Universität Bonn

Mitschrift zur Vorlesung

Randomisierte und Approximative Algorithmen

gehalten von

Prof. Dr. Thomas Kesselheim, Prof. Dr. Heiko Röglin

geTEXed von

Daniel Emse

Winter 2025/26

Anmerkungen und Korrekturen

Falls du Anmerkungen/Korrekturen hast, melde dich bei mir (`daniel.emse@uni-bonn.de`).

Inhaltsverzeichnis

Vorlesung **1**

I Randomisierte und Approximative Algorithmen **2**

1 Einleitung **3**

2 Greedy-Algorithmen **4**
2.1 Vertex Cover 4
2.2 Set Cover 5
2.3 Scheduling auf identischen Maschinen 6
2.4 Rucksackproblem 8

3 Runden und dynamische Programmierung **10**
3.1 Rucksackproblem 10
3.1.1 Dynamische Programmierung 10
3.1.2 Approximationsschema 11
3.2 Scheduling auf identischen Maschinen 12
3.2.1 Bin-Packing mit konstant vielen Objektgrößen 13
3.2.2 Approximationsschema 14
3.2.3 Untere Schranke für Approximierbarkeit 15

4 Randomisierte Approximationsalgorithmen **17**
4.1 Grundlagen der Wahrscheinlichkeitsrechnung 17
4.1.1 Diskrete Wahrscheinlichkeitsräume 17
4.1.2 Unabhängigkeit und bedingte Wahrscheinlichkeiten 18
4.2 Zufallsvariablen und Erwartungswerte 19
4.2.1 Analyse von RandMaxCut 22
4.2.2 Binomialverteilung und geometrische Verteilung 23
4.2.3 Vertex Cover 25
4.3 Max-SAT 26
4.3.1 Derandomisierung 27
4.3.2 Verbesserter Algorithmus 28

5 Lineare Programmierung und Runden **31**
5.1 Max-SAT 31
5.1.1 Randomisiertes Runden 31
5.1.2 Kombination beider Algorithmen 34
5.1.3 Nichtlineares randomisiertes Runden 35

Anhang **A**

Liste der Definitionen **B**

Liste der Aussagen **C**

Liste der Algorithmen **D**

Index **E**

Vorlesung

Teil I

Randomisierte und Approximative Algorithmen

Kapitel 1

Einleitung

Definition 1.1: Optimierungsproblem, Approximationsalgorithmus

Ein *Optimierungsproblem* Π besteht aus

- einer Meinge von Instanzen $\mathcal{I}_\Pi$
- für jede Instanz $I \in \mathcal{I}_\Pi$
  - einer Menge $\mathcal{S}_I$ von Lösungen
  - einer Zielfunktion $f_I : \mathcal{S}_I \to \mathbb{R}_{\geq 0}$.

Ein *Approximationsalgorithmus* A nimmt I als Eingabe und berechnet $A(I) \in \mathcal{S}_I$ (in polynomieller Zeit).
Sei $w_A(I) = f_I(A(I))$. Ein Approximationsalgorithmus erreicht Approximationsfaktor r, falls

$$w_A(I) \leq r \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Minimierungsprobleme)},$$

bzw.

$$w_A(I) \geq \frac{1}{r} \cdot \mathrm{OPT}(I) \quad \forall I \in \mathcal{I}_\Pi \qquad \text{(für Maximierungsprobleme)}.$$

Kapitel 2

Greedy-Algorithmen

2.1 Vertex Cover

[14.10.2025, Vorlesung 1]

Problem 2.1: Vertex Cover

Input: Ungerichteter Graph $G = (V, E)$
Output: Alle $V' \subseteq V$, sodass $\forall e = (u, v) \in E$ gilt $u \in V'$ oder $v \in V'$
Objective: Minimiere $|V'|$

Definition 2.2: Matching

Ein *Matching* $M \subseteq E$ eines Graphen $G = (V, E)$ ist eine Teilmenge der Kanten, sodass es keinen Knoten gibt, der zu mehr als einer Kante aus M inzident ist.
Es heißt *inklusionsmaximal*, wenn für alle Kanten $e \in E \setminus M$ gilt: $M \cup \{e\}$ ist kein Matching.

Algorithmus 2.3 MATCHING-VC

Input: ungerichteter Graph $G = (V, E)$
Output: Vertex Cover $V' \subseteq V$ mit $|V'| \leq 2 \cdot \mathrm{OPT}(G)$

1: ▷ *Berechne inklusionsmaximales Matching $M \subseteq E$:* ◁
2: $M := \emptyset$
3: **for all** $e = \{u, v\} \in E$ **do**
4: **if** weder u noch v ist inzident zu einer Kante in M **then**
5: $M := M \cup \{e\}$
6: $V(M) := \{v \in V \mid v \text{ ist inzident zu einer Kante in } M\}$
7: **return** $V(M)$

Satz 2.4: Matching-VC

Matching-VC hat Laufzeit $\mathcal{O}(|V| + |E|)$ für Graphen in Adjazenzlistendarstellung. Er berechnet ein Vertex Cover der Größe höchstens $2 \cdot \mathrm{OPT}(G)$, wobei $\mathrm{OPT}(G)$ die Größe des kleinsten Vertex Covers ist.

Beweis. **Laufzeit:** In Schritt 1 wird jede Kante genau einmal betrachtet, $\mathcal{O}(|V| + |E|)$.

In Schritt 2 wird jeder Knoten höchstens einmal betrachtet, $\mathcal{O}(|V|)$. Gemeinsam ergibt sich $\mathcal{O}(|V| + |E|)$.

Korrektheit: Angenommen, $e = (u, v) \in E$ wurde nicht von $V(M)$ abgedeckt. Das heißt, es wären weder $u \in V(M)$ noch $v \in V(M)$. Dann wäre $M \cup \{e\}$ ein Matching und e würde in Schritt 1 zu M hinzugefügt werden. Widerspruch zur Inklusionsmaximalität von M. Also ist $V(M)$ ein Vertex Cover.

Approximation: Sei V^* ein minimales Vertex Cover. V^* muss alle Kanten $e \in E$ abdecken, also auch alle $e \in M$.

Außerdem haben in M keine zwei Kanten denselben Endpunkt. Deshalb ist $|M| \leq |V^*|$.

Für unsere Lösung gilt $|V(M)| = 2 \cdot |M| \leq 2 \cdot \mathrm{OPT}(G)$.

□

2.2 Set Cover

Problem 2.5: Set Cover

Input: Menge S mit n Elementen, n Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^m S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: Alle $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$
Objective: Minimiere $\sum_{i \in A} c_i$

Algorithmus 2.6 GREEDY-SC

Input: Grundmenge S mit n Elementen, n Teilmengen $S_1, \ldots, S_m \subseteq S$ mit $\bigcup_{i=1}^m S_i = S$, Kosten $c_1, \ldots, c_m \in \mathbb{N}$.
Output: $A \subseteq \{1, \ldots, m\}$ mit $\bigcup_{i \in A} S_i = S$ und $\sum_{i \in A} c_i$ minimal

1: $A := \emptyset, C := \emptyset$
2: **while** $C \neq S$ **do**
3: Wähle S_i mit minimalen relativen Kosten $r_i(C) = \frac{c_i}{|S_i \setminus C|}$
4: Setze $\mathrm{price}(x) := r_i(C)$ für alle $x \in S_i \setminus C$
5: $A := A \cup \{i\}$
6: $C := C \cup S_i$
7: **return** A

Satz 2.7: Greedy-SC

GREEDY-SC ist ein H_n-Approximationsalgorithmus für Set Cover, wobei

$$H_n = \sum_{i=1}^{n} \frac{1}{i}$$

die n-te harmonische Zahl ist.

Untere Schranke für Greedy-SC: Sei $\varepsilon > 0$ sehr klein. Betrachte n Knoten und $m = n + 1$ Mengen, wobei $S_i = \{i\}$ mit Kosten $\frac{1}{n+1-i}$ ist (für $i \in S$) und $S_{n+1} = S$ mit Kosten $1 + \varepsilon$. Dann wählt GREEDY-SC wählt alle Einzelnen, OPT die gesamte Menge.

[14.10.2025, Vorlesung 1]
[15.10.2025, Vorlesung 2]

Beweis. Beobachtung: Es gilt

$$c(A) = \sum_{x \in S} \mathrm{price}(x).$$

Sei $x_1, \ldots, x_n$ die Reihenfolge der Elemente in S, wie sie vom GREEDY-SC abgedeckt werden (Beim Abdecken von mehreren Elementen in gleicher Iteration wähle beliebige Reihenfolge).

Lemma 2.8: Preis in Greedy-SC

Für alle $k \in \{1, \dots, n\}$ gilt

$$\text{price}(x_k) \leq \frac{\text{OPT}}{n-k+1}.$$

Beweis: Lemma 2.8. Betrachte ein beliebiges $k \in \{1, \dots, n\}$. Sei $i \in \{1, \dots, m\}$ der Index der Menge S_i, mit der x_k erstmalig abgedeckt wird. Bezeichne A die Auswahl der Mengen, *bevor* x_k abgedeckt wird.

Sei $C := \bigcup_{i \in A} S_i$. Es gilt $|C| \leq k-1$. Eine optimale Auswahl A^* von Mengen deckt ganz S ab, insbesondere auch $S \setminus C$, wobei $|S \setminus C| \geq n-k+1$.

Es gilt

$$\begin{aligned}\text{OPT} = \sum_{j \in A^*} c_j &\geq \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} c_j = \sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \underbrace{\frac{c_j}{|S_j \setminus C|}}_{= r_j(C) \geq r_i(C)} \\ &\geq r_i(C) \underbrace{\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C|}_{\geq |S \setminus C|} \geq \underbrace{r_i(C)}_{= \text{price}(x_k)} \cdot (n-k+1)\end{aligned}$$

wobei $r_j(C) \geq r_i(C)$ wegen der Greedy-Auswahl und

$$\sum_{\substack{j \in A^* \\ S_j \setminus C \neq \emptyset}} |S_j \setminus C| \geq |S \setminus C|,$$

weil A^* ganz $S \setminus C$ abdeckt. □

Nun gilt mit Lemma 2.8

$$c(A) = \sum_{k=1}^{n} \text{price}(x_k) \leq \sum_{k=1}^{n} \frac{\text{OPT}}{n-k+1} = \text{OPT} \sum_{i=1}^{n} \frac{1}{i} = \text{OPT} \cdot H_n$$

und Satz 2.7 ist gezeigt. □

2.3 Scheduling auf identischen Maschinen

Problem 2.9: Scheduling auf identischen Maschinen

Input: Menge $J = \{1, \dots, n\}$ von Jobs, Menge $M = \{1, \dots, m\}$ von Maschinen, Größen $p_j > 0$ für jeden Job $j \in J$

Output: Schedule $\pi : J \to M$, Last von Maschine i in Schedule π: $L_i(\pi) = \sum_{\substack{j \in J \\ \pi(j) = i}} p_j$,

Makespan $C(\pi) = \max_{i \in M} L_i(\pi)$

Objective: Finde Schedule π mit kleinstem Makespan

Algorithmus 2.10 LEASTLOADED

Input: Siehe Problem 2.9

Output: Siehe Problem 2.9

1: Betrachte Jobs in Reihenfolge $1, \dots, n$

2: Weise jedem Job der Maschine mit bislang kleinster Last zu

Satz 2.11: LeastLoaded

LeastLoaded ist ein $(2-\frac{1}{m})$-Approximationsalgorithmus für Scheduling auf identischen Maschinen.

Beweis. Sei π^* ein optimaler Schedule. Dann ist

$$C(\pi^*) \geq \frac{1}{m} \sum_{j \in J} p_j,$$

denn

$$C(\pi^*) = \max_{i \in M} \sum_{\substack{j \in J \\ \pi(j)=i}} p_j \geq \frac{1}{m} \sum_{i \in M} \sum_{\substack{j \in J \\ \pi(j)=i}} = \frac{1}{m} \sum_{j \in J} p_j.$$

Außerdem gilt $C(\pi^*) \geq \max_{j \in J} p_j$.

Sei π der Schedule, der LeastLoaded berechnet. Sei i die Maschine mit größter Last. Sei j der Job, der zuletzt zu i hinzugefügt wurde. Zu diesem Zeitpunkt war Maschine i diejenige mit der geringsten Last. Ihre Last war höchstens $\frac{1}{m}\sum_{k=1}^{j-1} p_k$. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m} \underbrace{\sum_{k=1}^{j-1} p_k}_{\leq \sum_{k \in J \setminus \{j\}} p_k} + p_j \leq \underbrace{\frac{1}{m} \sum_{k \in J} p_k}_{\leq C(\pi^*)} + \left(1 - \frac{1}{m}\right) \underbrace{p_j}_{\leq C(\pi^*)} \leq \left(2 - \frac{1}{m}\right) C(\pi^*)$$

und die Abschätzung ist gezeigt. □

Bemerkung 2.12. $2 - \frac{1}{m}$ ist eine untere Schranke für LeastLoaded.

Beweis. Sei $n = m(m-1)+1$ und $p_1 = \cdots = p_{m(m-1)} = 1$ und $p_{m(m-1)+1} = m$.

LeastLoaded führt auf jeder Maschine $m-1$ Jobs der Größe 1 durch. Auf einer Maschine wird zusätzlich ein Job der Größe m ausgeführt. Somit ist der Makespan $m-1+m = 2m-1$.

Eine optimale Belegung führt auf $m-1$ Maschinen je m Jobs der Größe 1 aus und auf der letzten Maschine einen Job der Größe m, wir haben also einen Makespan von m, und der Faktor stimmt. □

Algorithmus 2.13 LongestProcessingTime (LPT)

Input: Siehe Problem 2.9
Output: Siehe Problem 2.9

1: Sortiere Jobs so, dass $p_1 \geq p_2 \geq \cdots \geq p_n$.
2: Führe LeastLoaded auf den sortierten Jobs aus.

Satz 2.14: LPT

LongestProcessingTime ist ein $\frac{4}{3}$-Approximationsalgorithmus.

Beweis. Angenommen, es gäbe eine Instanz, sodass LPT einen Makespan von mehr als $\frac{4}{3} \cdot \mathrm{OPT}$ erreicht. Betrachte unter diesen Instanzen eine mit geringster Anzahl Jobs.

Sei i eine Maschine mit höchster Last in LPT-Schedule π. Sei j der letzte Job, der Maschine i zugewiesen wird. Es ist $j = n$, da sonst $p_1, \ldots, p_j$ ein Gegenbeispiel mit weniger Jobs wäre. Es gilt

$$C(\pi) = L_i(\pi) \leq \frac{1}{m} \sum_{k=1}^{n-1} p_k + p_n \leq \mathrm{OPT} + p_n.$$

Aus $C(\pi) > \frac{4}{3}\,\mathrm{OPT}$ folgt $p_n > \frac{1}{3}\,\mathrm{OPT}$. Da $p_1 \geq \cdots \geq p_n$ folgt $p_j > \frac{1}{3}\,\mathrm{OPT}$ für alle $j \in J$. Also erhält im optimalen Schedule jede Maschine höchstens zwei Jobs.

Sei k die Anzahl Jobs mit Größe $\frac{2}{3}\,\mathrm{OPT}$. Im optimalen Schedule sind k Jobs alleine auf einer Maschine und $n-k$ Jobs höchstens zu zweit.

Fall 1: $k = n$. Also ist $n \leq m$, Job n wird also einer leeren Maschine zugewiesen.

Fall 2: $k < n$. Zum Zeitpunkt bevor Job n hinzugefügt wird, hat LPT mindestens eine Maschine mit höchstens einem Job und dieser hat Größe höchstens $\frac{2}{3}\,\mathrm{OPT}$. Wegen $p_n \leq \frac{2}{3}\,\mathrm{OPT}$ ist die Last dieser Maschine im Anschluss höchstens $\frac{4}{3}\,\mathrm{OPT}$.

Beides ist ein Widerspruch zur Annahme. ↯ □

Bemerkung 2.15. $\frac{4}{3}$ ist eine untere Schranke für LONGESTPROCESSINGTIME.

Beweis. Übungsaufgabe. □

Wir werden später sehen, dass für alle $\varepsilon > 0$ ein $(1+\varepsilon)$-Approximationsalgorithmus existiert.

[15.10.2025, Vorlesung 2]
[21.10.2025, Vorlesung 3]

2.4 Rucksackproblem

Problem 2.16: Rucksackproblem (Knapsack Problem, KP)

Input: Kapazität $t \in \mathbb{N}$, Nutzen $p_1, \ldots, p_n \in \mathbb{N}$, Gewichte $w_1, \ldots, w_n \in \{1, \ldots, t\}$
Output: Teilmenge $A \subseteq \{1, \ldots, n\}$ mit $w(A) := \sum_{i \in A} w_i \leq t$
Objective: Maximiere $p(A) := \sum_{i \in A} p_i$

Algorithmus 2.17 GREEDY-KP

Input: Siehe Problem 2.16
Output: Siehe Problem 2.16

```
1: Sortiere Objekte nach Effizienz, sodass p_1/w_1 ≥ ··· ≥ p_n/w_n
2: B := ∅, i := 1
3: while i ≤ n and w(B) + w_i ≤ t do
4:     B := B ∪ {i}
5:     i := i + 1
6: Sei i* ein Objekt mit dem größten Nutzen
7: if p(B) < p_{i*} then
8:     A := {i*}
9: else A := B
10: return A
```

Satz 2.18: Greedy-KP

GREEDY-KP ist ein $\frac{1}{2}$-Approximationsalgorithmus.

Beweis. Sei $B := \{1, \ldots, i\}$ die im Algorithmus berechnete Menge. Falls $i = n$, gibt es nichts zu zeigen.

Falls $i < n$, gilt $w(B) + w_{i+1} > t$.

Behauptung: Es gilt $p(B) + p_{i+1} \geq \text{OPT}$:

Angenommen, $\text{OPT} > p(B) + p_{i+1}$. Dann gibt es eine Menge $A' \subseteq \{1, \ldots, n\}$, für die gilt, dass $w(A') \leq t < w(B) + w_{i+1}$ und $p(A') = \text{OPT} > p(B) + p_{i+1}$.

Betrachte $J := \{1, \ldots, i+1\} \setminus A'$ und $J' := A' \setminus \{1, \ldots, i+1\}$. Nun gilt

$$p(B) + p_{i+1} < p(A') = (p(B) + p_{i+1}) + p(J') - p(J) \implies p(J') > p(J)$$

und analog

$$w(B) + w_{i+1} > w(A') = (w(B) + w_{i+1}) + w(J') - w(J) \implies w(J') < w(J).$$

Sei $r = \frac{p_{i+1}}{w_{i+1}}$, dann

$$p(J) \geq r \cdot w(J), \qquad p(J') \leq r \cdot w(J').$$

Zusammen ergibt sich

$$p(J) \geq r \cdot w(J) > r \cdot w(J') \geq p(J') > p(J). \qquad \lightning$$

Damit ist die Behauptung gezeigt und es gilt $p(B) + p_{i+1} \geq \text{OPT}$.

Somit gilt

$$p(A) = \max\{p(B), p_{i^*}\} \geq \frac{1}{2}(p(B) + p_{i^*}) \geq \frac{1}{2}(p(B) + p_{i+1}) \geq \frac{1}{2}\,\text{OPT}$$

und die Aussage ist gezeigt. □

Kapitel 3

Runden und dynamische Programmierung

Definition 3.1: Approximationsschema, PTAS, FPATS

Ein *Approximationsschema* A für ein Optimierungsproblem Π ist ein Algorithmus, der zu jeder Eingabe (I, ε) mit $I \in \mathcal{I}_\Pi, \varepsilon > 0$ eine Lösung $A(I,\varepsilon) \in \mathcal{S}_I$ berechnet und es gilt

$$f(A(I,\varepsilon)) \leq (1+\varepsilon)\,\mathrm{OPT}(I) \qquad \text{bei Minimierungsproblemen, bzw.}$$
$$f(A(I,\varepsilon)) \geq (1-\varepsilon)\,\mathrm{OPT}(I) \qquad \text{bei Maximierungsproblemen.}$$

A heißt *polynomielles Approximationsschema* (PTAS), wenn die Laufzeit für jedes ε polynomiell in $|I|$ beschränkt ist.
A heißt *voll-polynomielles Approximationsschema* (FPTAS), wenn die Laufzeit polynomiell in $\frac{1}{\varepsilon}$ und $|I|$ ist.

Beispiel 3.2.

- $\Theta(|I|^{1/\varepsilon})$ ist PTAS, aber kein FPTAS
- $\Theta\left(\frac{|I|^{100}}{\varepsilon^{10}}\right)$ ist ein FPTAS

3.1 Rucksackproblem

3.1.1 Dynamische Programmierung

Sei $W(I,p)$ *geringste* Gewicht unter allen Teilmengen von $\{1,\dots,i\}$, die Nutzen mindestens p erreichen, bzw. ∞, wenn das nicht möglich ist.

Falls $I \subseteq \{1,\dots,i\}$ gilt: Falls $\sum_{i\in I} p_i \geq p$, dann ist $\sum_{i\in I} w_i \geq W(I,p)$.

Es gilt

$$W(0,p) = \begin{cases} 0 & p \leq 0 \\ \infty & p > 0, \end{cases}$$

$$W(1,p) = \begin{cases} 0 & p \leq 0 \\ w_1 & 0 < p \leq p_1 \\ \infty & p > p_1. \end{cases}$$

Sei $P := \max_{\{1,\dots,n\}} p_i$. Uns interessieren $W(i,p)$ für $i = 0,\dots,n$ und $p = 0,\dots,nP$.

Rekursionsformel für $W(i,p)$:

Sei $I \subseteq \{1,\dots,i\}$ diejenige Teilmenge mit minimalem Gewicht unter allen mit $\sum_{j\in I} p_j \geq p$.

Fall 1: $i \notin I$. Dann gilt $W(i,p) = W(i-1,p)$.

Fall 2: $i \in I$. Dann betrachte $I \setminus \{i\}$. Es ist $W(I \setminus \{i\}) = W(i,p) - w(i)$ und $p(I \setminus \{i\}) \geq p - p_i$. Daraus folgt

$$W(i-1, p-p_i) \leq W(i,p) - w_i.$$

Umgekehrt sei $I' \subseteq \{1,\dots,i-1\}$ diejenige Menge, die $W(i-1,p-p_i)$ liefert. Nun gilt

$$W(I' \cup \{i\}) = W(i-1,p-p_i) + w_i, \qquad p(I' \cup \{i\}) = p(I') + p_i \geq p.$$

Daraus folgt, dass $W(i,p) \leq W(i-1,p-p_i) + w_i$.

Umstellen liefert

$$W(i,p) = W(i-1,p-p_i) + w_i.$$

Insgesamt ergibt sich die Rekursion

$$W(i,p) = \min\{W(i-1,p), W(i-1,p-p_i) + w_i\}, \qquad W(0,p) = \begin{cases} \infty & p > 0 \\ 0 & p \leq 0. \end{cases}$$

Algorithmus 3.3 DYNKP

Input: Siehe Problem 2.16
Output: Siehe Problem 2.16

1: $P := \max_{i \in \{1,\dots,n\}} p_i$
2: **for** $i = 1$ **to** n **do**
3: **for** $p = 1$ **to** nP **do**
4: $W(i,p) = \min\{W(i-1,p), W(i-1,p-p_i) + w_i\}$
5: **return** maximales $p \in \{1,\dots,nP\}$ mit $W(n,p) \leq t$.

Satz 3.4: DynKP

DYNKP liefert in Zeit $\Theta(n^2 P)$ den maximal erreichbaren Nutzen.

3.1.2 Approximationsschema

Algorithmus 3.5 FPTAS-KP

Input: Siehe Problem 2.16, $\varepsilon > 0$
Output: Siehe Problem 2.16

1: $P := \max_{i \in 1,\dots,n} p_i$
2: $K := \frac{\varepsilon P}{n}$
3: **for** $i = 1$ **to** n **do**
4: $p'_i = \lfloor \frac{p_i}{K} \rfloor$
5: Benutze DYNKP auf $p'_1,\dots,p'_n, w_1,\dots,w_n, t$

Satz 3.6: FPTAS-KP

FPTAS-KP hat eine Laufzeit von $\mathcal{O}(\frac{n^3}{\varepsilon})$ und berechnet eine $1-\varepsilon$-Approximation.

Beweis.

Laufzeit: DYNKP braucht Zeit $\Theta(n^2 P')$, wobei $P' = \max_{i \in \{1,\dots,n\}} p'_i$. Es gilt für alle i:

$$p'_i = \left\lfloor \frac{p_i}{K} \right\rfloor \leq \left\lfloor \frac{P}{K} \right\rfloor = \left\lfloor \frac{n}{\varepsilon} \right\rfloor \leq \frac{n}{\varepsilon},$$

also ist auch $P' \leq \frac{n}{\varepsilon}$.

Approximationsfaktor: Sei I' eine optimale Lösung auf $p'_1, \ldots, p'_n$ und I eine optimale Lösung auf $p_1, \ldots, p_n$.

I' ist auch eine optimale Lösung auf $\{p_i^*\} := \{K \cdot p'_i\}$.

Beispiel 3.7. Für $n = 4, P = 50, \varepsilon = \frac{4}{5}, K = \frac{\varepsilon P}{n} = 10$. Für $p_1 = 33$ erhalten wir $p'_1 = 3$ bzw. $p_1^* = 30$.

Es gilt $p_i^* = K \cdot \lfloor \frac{p_i}{K} \rfloor \geq K \cdot (\frac{p_i}{K} - 1) = p_i - K$ und $p_i^* \leq K \cdot \frac{p_i}{K} = p_i$, also $p_i^* \in [p_i - K, p_i]$.

Für alle $J \subseteq \{1, \ldots, n\}$ gilt

$$p^*(J) := \sum_{i \in J} p_i^* \leq \sum_{i \in J} p_i = p(J)$$

und

$$p^*(J) = \sum_{i \in J} p_i^* \geq \sum_{i \in J} (p_i - K) = \sum_{i \in J} p_i - |J| \cdot K \geq p(J) - nK.$$

Damit ist $p(I') \geq p^*(I') \geq p^*(I) \geq p(I) - nK$, da I' optimal hinsichtlich p^* ist.

Weiterhin ist $p(I) \geq P$, also gilt

$$\frac{p(I')}{\mathrm{OPT}} \geq \frac{p(I) - nK}{p(I)} \geq 1 - \frac{nK}{P} = 1 - \varepsilon$$

und wir erhalten $1 - \varepsilon$ als Approximationsfaktor.

□

[21.10.2025, Vorlesung 3]
[22.10.2025, Vorlesung 4]

3.2 Scheduling auf identischen Maschinen

Erster Ansatz für Approximationsschema: Wir nennen einen Job $j \in J$ *klein*, falls

$$p_j \leq \varepsilon \cdot \frac{1}{m} \sum_{k=1}^{n} p_k,$$

sonst heißt er *groß*. Wir betrachten folgenden Algorithmus:

Algorithmus 3.8 GrossKleinScheduling

Input: Siehe Problem 2.9, $\varepsilon > 0$
Output: Siehe Problem 2.9

1: Berechne optimalen Schedule auf großen Jobs ▷ *Bspw. durch Ausprobieren aller Schedules*
2: Ergänze kleine Jobs mit LeastLoaded

Satz 3.9: GroßKleinScheduling

GrossKleinScheduling ist ein PTAS mit Laufzeit $m^{\frac{m}{\varepsilon}}$.

Beweis. **Laufzeit:** Für Algorithmus 3.8 gibt es höchstens $\frac{m}{\varepsilon}$ große Jobs, denn jeder ist mindestens $\varepsilon \frac{1}{m} \sum_{k=1}^{n} p_k$ groß. Damit gibt es maximal $m^{\frac{m}{\varepsilon}}$ mögliche Schedules. Damit hat Algorithmus 3.8 polynomielle Laufzeit für jedes $\varepsilon > 0$, wenn m konstant ist.

Approximationsfaktor: Betrachte die Maschine mit der größten Last.

Fall 1: Diese Maschine erhält nur große Jobs in unserem Schedule. Dann ist der Makespan durch die kleinen Jobs unverändert. Unser Schedule ist optimal auf den großen Jobs, und somit auch optimal auf allen Jobs.

Fall 2: Diese Maschine erhält auch einen kleinen Job. Im Moment der Zuweisung des letzten solchen Jobs durch LEASTLOADED ist die Last höchstens die $\frac{1}{m}\sum_{k=1}^{n} p_k$. Der letzte Job ist ein kleiner Job, somit haben wir am Ende eine Gesamtlast von höchstens

$$(1+\varepsilon)\frac{1}{m}\sum_{k=1}^{n} p_k \leq (1+\varepsilon)\,\mathrm{OPT}\,.$$

□

3.2.1 Bin-Packing mit konstant vielen Objektgrößen

Problem 3.10: Bin-Packing

Input: Objektgrößen $s_1, \ldots, s_n \in \mathbb{N}$, Kapazität $T \in \mathbb{N}$
Output: Partitionen $C_1, \ldots, C_\ell$ der Menge $\{1, \ldots, n\}$ mit $\sum_{j \in C_i} s_j \leq T$ für alle i
Objective: Minimiere ℓ

Spezialfall: Wir wählen als Eingabe $(s_1, \ldots, s_z) \in \mathbb{N}^z, (n_1, \ldots, n_z) \in \mathbb{N}^z, T \in \mathbb{N}$. Dabei haben wir jeweils n_i Objekte der Größe s_i.

Wir bezeichnen mit $\mathrm{BINS}(i_1, \ldots, i_z)$ den optimalen Zielfunktionswert, wenn es $i_j \leq n_j$ Objekte der Größe s_j gibt. Uns interessiert $\mathrm{BINS}(n_1, \ldots, n_z)$. Berechne aller Werte $\mathrm{BINS}(i_1, \ldots, i_z)$ für $(i_1, \ldots, i_z) \in \mathbb{N}_0^z$ mit $i_j \leq n_j$. Wenn z konstant ist, gibt es höchstens $(n+1)^z \in \mathcal{O}(n^z)$ viele solche Vektoren.

Definiere

$$C := \left\{ (i_1, \ldots, i_z) \in \mathbb{N}_0^z \;\middle|\; \sum_{j=1}^{z} i_j s_j \leq T \right\} \setminus \{(0, \ldots, 0)\}.$$

Wir bemerken:

- $\mathrm{BINS}(0, \ldots, 0) = 0$
- $\mathrm{BINS}(i_1, \ldots, i_z) = 1$ für $(i_1, \ldots, i_z \in C)$

Für die allgemeine Größe von $\mathrm{BINS}(i_1, \ldots, i_z)$ betrachte ein optimales Packing. Der erste Bin erhalte $(c_1, \ldots, c_z) \in C$ viele Objekte. So erhalten die übrigen Bins insgesamt $(i_1 - c_1, \ldots, i_z - c_z)$ Objekte und es gilt

$$\mathrm{BINS}(i_1, \ldots, i_z) = 1 + \mathrm{BINS}(i_1 - c_1, \ldots, i_z - c_z).$$

Als allgemeine Rekursionsformel ergibt sich

$$\mathrm{BINS}(i_1, \ldots, i_n) = 1 + \min_{(c_1, \ldots, c_z) \in C} \mathrm{BINS}(i_1 - c_1, \ldots, i_z - c_z) \qquad \text{für alle } (i_1, \ldots, i_z) \neq (0, \ldots, 0)$$

und $\mathrm{BINS}(0, \ldots, 0) = 0$.

Wir berechnen Werte von $\mathrm{BINS}(i_1, \ldots, i_z)$ aufsteigend nach $\sum_{j=1}^{z} i_j$. Insgesamt werden $\mathcal{O}(n^z)$ viele Vektoren mit jeweils Laufzeit $\mathcal{O}(n^z)$ pro Vektor, damit ergibt sich eine Gesamtlaufzeit von $\mathcal{O}(n^{2z})$, also folgender Satz:

Satz 3.11: Bin-Packing

Das Bin-Packing-Problem mit z verschiedenen Objektgrößen kann für Instanzen mit n Objekten in Zeit $\mathcal{O}(n^{2z})$ gelöst werden (wobei die Konstante in der $\mathcal{O}$-Notation von z abhängt.)

3.2.2 Approximationsschema

Zusätzliche Eingabe: T. Der Algorithmus liefert entweder einen Schedule mit Makespan höchstens $(1+\varepsilon)T$ oder er beweist, dass $\mathrm{OPT} > T$.

Wir bezeichnen einen Job j als *groß*, falls $p_j > \varepsilon T$. Wir runden die Größen aller großen Jobs auf Vielfache von $\varepsilon^2 T$ ab, somit gibt es höchstens $\frac{1}{\varepsilon^2}$ viele Jobgrößen. Auf diesen bestimmen wir eine optimale Bin-Packing-Lösung. Wir erhalten als Antwort entweder eine Packung der großen Jobs auf m Maschinen mit Makespan $\leq T$ (bzgl. der gerundeten Jobgrößen), oder dass es nicht möglich ist (dann brechen wir ab). Für die kleinen Jobs verwenden wir anschließend LeastLoaded.

Für den Makespan gilt insgesamt:

- Erhält eine Maschine einen kleinen Job, betrachte den letzten derartigen Job. Dann gilt

$$L_i(\pi) \leq \underbrace{\left(\frac{1}{m}\sum_{k=1}^{n} p_k\right)}_{\leq T} + p_j \leq (1+\varepsilon)T.$$

- Erhält eine Maschine keinen kleinen Job, dann ist die Last durch die gerundeten Jobgrößen höchstens T. Sei p'_j die abgerundete Jobgröße und p_j die tatsächliche Jobgröße. Dann gilt

$$p_j \geq \varepsilon T, \qquad p'_j \geq p_j - \varepsilon^2 T \geq p_j - \varepsilon p'_j \implies p_j \leq (1+\varepsilon)p'_j,$$

  unter der Voraussetzung, dass $\frac{1}{\varepsilon} \in \mathbb{N}$.

Somit sind nicht gerundete Jobgrößen höchstens um den Faktor $(1+\varepsilon)$ größer und somit ist die Last der Maschine $\leq (1+\varepsilon)T$ bezüglich der tatsächlichen Jobgrößen.

Hier der Algorithmus nochmal in Pseudocode:

Algorithmus 3.12 $B_\varepsilon(T)$

1: Sei $J' = \{j \in J \mid p_j > \varepsilon T\}$ die Menge der großen Jobs
2: Für $j \in J$ sei p'_j der Wert p_j auf das nächste Vielfache von $\varepsilon^2 T$ abgerundet. Sei I' die Instanz, die nur aus Jobs mit den Größen p'_j mit $j \in J'$ besteht.
3: Konstruiere eine Instanz für Bin-Packing mit Objektgrößen $\{p'_j \mid j \in J'\}$ und Kapazität T. Bezeichne a die minimale Anzahl an Bins, die für diese Instanz benötigt werden. Berechne a und die dazugehörige Verteilung π' der Objekte mit Hilfe des dynamischen Programms aus Satz 3.11.
4: **if** $a > m$ **then**
5: | **return** $\mathrm{OPT}(I) \geq \mathrm{OPT}(I') > T$
6: **else**
7: | Erweitere π' auf alle Jobs aus J durch Hinzufügen der Jobs aus $J \setminus J'$ mittels LeastLoaded.
8: | **return** resultierenden Schedule π

Somit haben wir gezeigt:

Lemma 3.13: $B_\varepsilon(T)$

Es gibt einen Algorithmus, der in Zeit $\mathcal{O}(n^{\frac{2}{\varepsilon^2}})$ entweder beweist, dass es keinen Schedule mit Makespan T gibt oder einen Schedule mit Makespan $(1+\varepsilon)T$ berechnet.

Für einen Optimalen Makespan gilt num

$$\begin{aligned}\text{OPT} &\geq \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil \\ \text{OPT} &\geq p_{\max} \\ \text{OPT} &\leq \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil + p_{\max}.\end{aligned}$$

Setze initial $L := \max\left\{\left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil, p_{\max}\right\}$ und $U := \left\lceil \frac{1}{m}\sum_{k=1}^{n} p_k \right\rceil + p_{\max}$. Dabei haben wir folgende Invarianten:

1. $L \leq \text{OPT}$
2. Wir können einen Schedule mit Makespan höchstens $(1+\varepsilon)U$ in Zeit $\mathcal{O}(n^{\frac{2}{\varepsilon^2}})$ berechnen.

Nun berechne $T := \left\lfloor \frac{L+U}{2} \right\rfloor$ und prüfe, ob ein Schedule mit Makespan T berechnet werden kann. Falls ja, setze $U := T$, sonst setze $L := T+1$. Es gilt $U - L \leq \frac{p_{\max}}{2^i}$ nach i Iterationen, somit gilt $U - L \leq \frac{1}{2}$ nach $\lfloor \log_2(p_{\max}) \rfloor + 1$ Iterationen. Dann muss $L = U$ gelten und wir erhalten als Resultat folgenden Satz:

Satz 3.14: PTAS Scheduling

Für Scheduling auf identischen Maschinen existiert ein PTAS mit einer Laufzeit von $\mathcal{O}(m^{\frac{2}{\varepsilon^2}} \log(p_{\max}))$.

[22.10.2025, Vorlesung 4]
[28.10.2025, Vorlesung 5]

3.2.3 Untere Schranke für Approximierbarkeit

Definition 3.15: Stark NP-schwer

Sei Π ein Optimierungsproblem, in dessen Instanzen ganze Zahlen enthalten sind. Π heißt *stark NP-schwer*, wenn es ein Polynom q gibt, sodass Π eingeschränkt auf Instanzen I, in denen alle Zahlen durch $q(|I|)$ beschränkt sind, bereits NP-schwer ist.

Scheduling auf identischen Maschinen ist tatsächlich stark NP-schwer.

Satz 3.16: Scheduling hat keinen FPTAS

Unter der Annahme $\text{P} \neq \text{NP}$ existiert kein FPTAS für Scheduling auf identischen Maschinen.

Beweis. Angenommen, es gäbe einen FPTAS. Weil Scheduling auf identischen Maschinen stark NP-schwer ist, gibt es ein Polynom q, sodass es bereits NP-schwer ist, das Problem mit Jobgrößen $p_1 \leq \cdots \leq p_n \in \{1, \ldots, q(n)\}$ zu lösen.

Berechne mit dem FPTAS eine $(1+\varepsilon)$-Approximation, wobei $\varepsilon = \frac{1}{2nq(n)}$. Die Laufzeit ist polynomiell in n und $\frac{1}{\varepsilon} = 2nq(n)$, also beides polynomiell in n. Für den Approximationsfaktor

erhalten wir $w_A(I, \varepsilon) \leq (1 + \varepsilon)\,\mathrm{OPT}$. Es gilt

$$\mathrm{OPT}(I) \leq \sum_{i=1}^{n} p_i \leq n \cdot q(n)$$

und folglich

$$(1 + \varepsilon)\,\mathrm{OPT} = \mathrm{OPT} + \underbrace{\varepsilon\,\mathrm{OPT}}_{\leq \frac{1}{2}} \leq \mathrm{OPT}(I) + \varepsilon n q(n) = \mathrm{OPT} + \frac{1}{2}.$$

Weil alle Jobgrößen ganzzahlig sind, sind auch $w_A(I, \varepsilon)$ und $\mathrm{OPT}(I)$ ganze Zahlen. Da die Differenz höchstens $\frac{1}{2}$ ist, ist $w_A(I, \varepsilon) = \mathrm{OPT}(I)$. Also löst der FPTAS das Problem exakt in polynomieller Zeit. Also muss P = NP sein. $\lightning$ zur Annahme □

Kapitel 4

Randomisierte Approximationsalgorithmen

Problem 4.1: Maximum-Cut Problem

Input: Ungerichteter Graph $G = (V, E)$, Kantengewichte $w : E \to \mathbb{R}_{>0}$
Output: Menge $U \subseteq V$. Wir nennen $(U, V \setminus U)$ *Schnitt* des Graphen G
Objective: Maximiere $\sum_{x \in U} \sum_{y \in V \setminus U} w(x, y)$ mit $w(x, y) = 0$, falls $\{x, y\} \notin E$.

Algorithmus 4.2 RandMaxCut

Input: Wie in Problem 4.1
Output: Wie in Problem 4.1

```
1: U := ∅
2: for v ∈ V do
3:     Wirf eine Münze
4:     if Münze zeigt Kopf then
5:         Füge v zu U hinzu
6: return U
```

4.1 Grundlagen der Wahrscheinlichkeitsrechnung

4.1.1 Diskrete Wahrscheinlichkeitsräume

Definition 4.3: Wahrscheinlichkeitsraum

Sei Ω eine endliche oder abzählbare Menge (*Ergebnismenge*).
Elemente von Ω heißen *Elementarereignisse*, Teilmengen von Ω heißen *Ereignisse*.
Für $A \subseteq \Omega$ bezeichne $\overline{A} := \Omega \setminus A$ das *Gegenereignis*.
Ein(-e) *Wahrscheinlichkeit(-smaß)* ist eine Funktion $\mathbb{P} : 2^\Omega \to [0, 1]$ mit $\mathbb{P}[\Omega] = 1$ und $\mathbb{P}$ *σ-additiv*, d.h. für $A_1, A_2, \ldots \subseteq \Omega$ mit $A_i \cap A_j = \emptyset$ für $i \neq j$ gilt

$$\mathbb{P}\left[\bigcup_{i=1}^{\infty} A_i\right] = \sum_{i=1}^{\infty} \mathbb{P}[A_i].$$

Das Paar $(\Omega, \mathbb{P})$ heißt (diskreter) *Wahrscheinlichkeitsraum*.

Es reicht, $\mathbb{P}[a]$ für alle $a \in \Omega$ zu definieren, denn

$$\mathbb{P}[\{a_1, \ldots, a_n\}] = \sum_{i=1}^{n} \mathbb{P}[\{a_i\}].$$

Beispiel 4.4. Bei einem Würfelwurf setze $\Omega = \{1, \ldots, 6\}$. Dann ist $\mathbb{P}[i] = \frac{1}{6}$ für alle $i \in \Omega$. Es ist auch $\mathbb{P}[\text{„gerade Zahl“}] = \mathbb{P}[\{2, 4, 6\}] = 3 \cdot \frac{1}{6} = \frac{1}{2}$.

Wir modellieren nun einen Wahrscheinlichkeitsraum für RandMaxCut:

Sei $V = \{v_1, \dots, v_n\}$. Setze $\Omega = \{0,1\}^n$. Wir berechnen den Schnitt als

$$U = \{v_i \in V \mid \omega_i = 1\}.$$

Für alle $e = \{v_i, v_j\} \in E$, ist die Wahrscheinlichkeit, dass e über den Schnitt läuft, gerade $\frac{1}{2}$, denn es ist

$$\mathbb{P}[e \text{ läuft über den Schnitt } (U, V \setminus U)] = \mathbb{P}[\{\omega \in \Omega \mid \omega_i \neq \omega_j\}] = \frac{2^{n-1}}{2^n} = \frac{1}{2}.$$

4.1.2 Unabhängigkeit und bedingte Wahrscheinlichkeiten

Definition 4.5: Unabhängige Ereignisse

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. Zwei Ereignisse $A, B \subseteq \Omega$ heißen *unabhängig*, falls

$$\mathbb{P}[A \cap B] = \mathbb{P}[A] \cdot \mathbb{P}[B].$$

Allgemeiner: Ereignisse $A_1, \dots, A_k \subseteq \Omega$ heißen *unabhängig*, wenn für alle $I \subseteq \{1, \dots, k\}$ gilt

$$\mathbb{P}\left[\bigcap_{i \in I} A_i\right] = \prod_{i \in I} \mathbb{P}[A_i].$$

Beispiel 4.6.

- Sei $\Omega = \{(i,j) \mid i,j \in \{1,\dots,6\}\}$. Dann ist $\mathbb{P}[(i,j)] = \frac{1}{36}$.

  Seien $A :=$ „Erster Würfel zeigt eine gerade Zahl“ und $B :=$ „Zweiter Würfel zeigt ≥ 3“. Dann ist $\mathbb{P}[A] = \frac{1}{2}, \mathbb{P}[B] = \frac{2}{3}$ und $\mathbb{P}[A \cap B] = \frac{1}{3} = \mathbb{P}[A] \cdot \mathbb{P}[B]$. Also sind A und B unabhängig.
- Für eine endlihce Folge von Wahrscheinlichkeitsräumen $(\Omega_1, \mathbb{P}_1), \dots, (\Omega_n, \mathbb{P}_n)$ definieren wir den *Produktraum* durch $\Omega := \Omega_1 \times \cdots \times \Omega_n$ und $\mathbb{P}[(x_1, \dots, x_n)] := \mathbb{P}_1[x_1] \cdot \ldots \cdot \mathbb{P}_n[x_n]$.

In RandMaxCut ist der Wahrscheinlichkeitsraum eigentlich ein Produktraum von n Münzwürfen $(\{0,1\}, \mathbb{P})$ mit $\mathbb{P}[0] = \mathbb{P}[1] = \frac{1}{2}$. Seien $e, e' \in E$ ohne gemeinsame Endpunkte. Dann ist

$$\mathbb{P}[\text{„}e \text{ und } e' \text{ laufen über Schnitt“}] = \frac{1}{4} = \mathbb{P}[\text{„}e \text{ läuft über Schnitt“}] \cdot \mathbb{P}[\text{„}e' \text{ läuft über Schnitt“}].$$

Definition 4.7: Bedingte Wahrscheinlichkeit

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. Seien $A, B \subseteq \Omega$ mit $\mathbb{P}[B] > 0$. Dann bezeichne

$$\mathbb{P}[A \mid B] := \frac{\mathbb{P}[A \cap B]}{\mathbb{P}[B]}$$

die *bedingte Wahrscheinlichkeit* von A gegeben B.

Beispiel 4.8. Wir betrachten zwei unanbhängige Würfelwürfe. Sei $A :=$ „Erster Würfel zeigt 6“ und $B :=$ „Summe beider Würfe ist 10“. Dann sind

$$\mathbb{P}[A] = \frac{1}{6}, \quad \mathbb{P}[B] = \frac{3}{36}, \quad \mathbb{P}[A \cap B] = \frac{1}{36} \quad \text{und} \quad \mathbb{P}[A \mid B] = \frac{1}{3}.$$

4.2 Zufallsvariablen und Erwartungswerte

Definition 4.9: Zufallsvariable

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum. eine Abbildung $X : \Omega \to \mathbb{R}$ heißt (diskrete, reelle) *Zufallsvariable*. Wir schreiben

$$\mathbb{P}[X = a] = \mathbb{P}[\{\omega \in \Omega \mid X(\omega) = a\}]$$

und allgemeiner

$$\mathbb{P}[X \in A] = \mathbb{P}[\{\omega \in \Omega \mid X(\omega) \in A\}].$$

Beispiel 4.10. Betrachte zwei unabhängige Würfelwürfe. Sei $X : \Omega := \{1, \dots, 12\}$ definiert durch $X(\omega) := \omega_1 + \omega_2$ für alle $\omega = (\omega_1, \omega_2) \in \Omega$.

Definition 4.11: Unabhängigkeit (Zufallsvariablen)

Seien $X, Y : \Omega \to \mathbb{R}$ Zufallsvariablen. X und Y heißen *unabhängig*, wenn für alle $x, y \in \mathbb{R}$ gilt

$$\mathbb{P}[(X = x) \cap (Y = y)] = \mathbb{P}[X = x] \cdot \mathbb{P}[Y = y].$$

Allgemeiner seien $X_1, \dots, X_n : \Omega \to \mathbb{R}$ Zufallsvariablen. Diese heißen *unabhängig*, wenn für alle $I \subseteq \{1, \dots, n\}, x_i \in \mathbb{R}$ gilt

$$\mathbb{P}\left[\bigcap_{i \in I} (X_i = x)\right] = \prod_{i \in I} \mathbb{P}[X_i = x].$$

Definition 4.12: Erwartungswert

Sei $X : \Omega \to \mathbb{R}$ eine diskrete Zufallsvariable. Falls die Reihe

$$\sum_{\omega \in \Omega} |X(\omega)| \cdot \mathbb{P}[\omega]$$

konvergiert, heißt

$$\mathbb{E}[X] := \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\omega]$$

Erwartungswert von X.

[28.10.2025, Vorlesung 5]
[29.10.2025, Vorlesung 6]

Sei X eine Zufallsvariable, für die der Erwartungswert gemäß Definition 4.12 existiert. Bezeichnen wir mit $R = X(\Omega)$ die abzählbare Teilmenge der reelen Zahlen, die von X angenommen werden können, können wir den Erwartungswert auch schreiben als

$$\mathbb{E}[X] = \sum_{x \in R} x \cdot \mathbb{P}[X = x].$$

Beispiel 4.13.

- Für die Zufallsvariable X, die die Summe zweier unabhängiger Würfelwürfe modelliert, ist

$$\mathbb{E}[X] = \sum_{x=2}^{12} x \cdot \mathbb{P}[X = x] = \frac{1}{36}(2+2\cdot 3+3\cdot 4+4\cdot 5+5\cdot 6+6\cdot 7+5\cdot 8+4\cdot 9+3\cdot 10+2\cdot 11+12) = 7.$$

- Für die Zufallsvariable X, die beschreibt, wie oft in zehn Münzwürfen „Zahl“ geworfen wird, ist es anhand der Definition schon schwieriger, den Erwartungswert zu berechnen. Mit Mühe erhält man

$$\mathbb{E}[X] = 0 \cdot \frac{1}{1024} + 1 \cdot \frac{10}{1024} + 2 \cdot \frac{45}{1024} + \cdots + 10 \cdot \frac{1}{1024} = 5.$$

- Wieder betrachten wir den Algorithmus RANDMAXCUT (Algorithmus 4.2) auf dem obigen Beispiel, in dem die drei Kanten e_1, e_2 und e_3 ein Dreieck bilden. Sei wieder X die Zufallsvariable, die die Anzahl an Kanten beschreibt, die über den Schnitt laufen. Da X nur die Werte 0 oder 2 annehmen kann, gilt

$$\mathbb{E}[X] = 0 \cdot \mathbb{P}[X = 0] + 2 \cdot \mathbb{P}[X = 2] = 2 \cdot \frac{3}{4} = \frac{3}{2}.$$

Im obigen Beispiel für RANDMAXCUT haben wir die erwartete Anzahl an Kanten, die über den Schnitt laufen, ausgerechnet. Für Graphen mit vielen Knoten und Kanten wäre eine ähnliche Rechnung sehr aufwändig, da wir in diesem Beispiel alle möglichen Realisierungen einzeln betrachtet haben. Um das Ausrechnen von Erwartungswerten zu vereinfachen, lernen wir einige wichtige Rechenregeln kennen.

Satz 4.14: Linearität des Erwartungswerts

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und seien $X, Y : \Omega \to \mathbb{R}$ Zufallsvariablen mit existierenden Erwartungswerten. Dann gilt:

(a) *Für $c, d \in \mathbb{R}$ existiert der Erwartungswert von $(cX + d) : \Omega \to \mathbb{R}$ mit $(cX + d)(\omega) := c \cdot X(\omega) + d$ für alle $\omega \in \Omega$. Es gilt $\mathbb{E}[cX + d] = c \cdot \mathbb{E}[X] + d$.*

(b) *Der Erwartungswert der Zufallsvariablen $(X + Y) : \Omega \to \mathbb{R}$ mit $(X + Y)(\omega) := X(\omega) + Y(\omega)$ für alle $\omega \in \Omega$ existiert und es gilt $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$.*

(c) *Sind X und Y unabhängig, so existiert der Erwartungswert der Zufallsvariablen $(XY) : \Omega \to \mathbb{R}$ mit $(XY)(\omega) := X(\omega) \cdot Y(\omega)$ für alle $\omega \in \Omega$ und es gilt $\mathbb{E}[XY] = \mathbb{E}[X] \cdot \mathbb{E}[Y]$.*

Beweis.

(a): Aus Definition 4.12 folgt

$$\begin{aligned}\mathbb{E}[cX + d] &= \sum_{\omega \in \Omega} (cX(\omega) + d) \cdot \mathbb{P}[\{\omega\}] \\ &= c \cdot \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\{\omega\}] + d \cdot \sum_{\omega \in \Omega} \mathbb{P}[\{\omega\}] \\ &= c \cdot \mathbb{E}[X] + d.\end{aligned}$$

Die Existenz des Erwartungswerts von $cX + d$ folgt aus der Existenz des Erwartungswerts von X.

(b): Es gilt, auch mit Definition 4.12:

$$\begin{aligned}\mathbb{E}[X + Y] &= \sum_{\omega \in \Omega} (X(\omega) + Y(\omega)) \cdot \mathbb{P}[\{\omega\}] \\ &= \sum_{\omega \in \Omega} X(\omega) \cdot \mathbb{P}[\{\omega\}] + \sum_{\omega \in \Omega} Y(\omega) \cdot \mathbb{P}[\{\omega\}] \\ &= \mathbb{E}[X] + \mathbb{E}[Y].\end{aligned}$$

(c): Seien $R = X(\Omega)$ und $S = Y(\Omega)$. Dann gilt die abzählbaren Mengen von Werten, die X

und Y annehmen können. Es folgt mit Definition 4.12:

$$\begin{aligned}\mathbb{E}[XY] &= \sum_{\omega\in\Omega} X(\omega)\cdot Y(\omega)\cdot \mathbb{P}[\{\omega\}] \\ &= \sum_{x\in R}\sum_{y\in S} x\cdot y\cdot \mathbb{P}[(X=x)\cap(Y=y)] \\ &= \sum_{x\in R}\sum_{y\in S} x\cdot y\cdot \mathbb{P}[X=x]\cdot\mathbb{P}[Y=y] \\ &= \left(\sum_{x\in R} x\cdot\mathbb{P}[X=x]\right)\cdot\left(\sum_{y\in S} y\cdot\mathbb{P}[Y=y]\right) \\ &= \mathbb{E}[X]\cdot\mathbb{E}[Y],\end{aligned}$$

wobei wir in der dritten Gleichheit die Unabhängigkeit von X und Y verwendet haben. Die Existenz des Erwartungswerts von XY folgt aus der Existenz der Erwartungswerte von X und Y.

□

Satz 4.15: Markov

Sei $(\Omega,\mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und sei $X:\Omega\to\mathbb{R}_{\geq 0}$ eine Zufallsvariable mit existierendem Erwartungswert. Dann gilt für alle $a>0$:

$$\mathbb{P}[X\geq a]\leq\frac{\mathbb{E}[X]}{a}.$$

Wir können die Markov-Ungleichung umformulieren, um eine noch einprägsamere Form zu erhalten. Sei $X:\Omega\to\mathbb{R}_{\geq 0}$ eine Zufallsvariable. Dann gilt für alle $a\geq 1$

$$\mathbb{P}[X\geq a\cdot\mathbb{E}[X]]\leq\frac{1}{a}.$$

Beweis von Satz 4.15. Es gilt

$$\mathbb{E}[X]=\sum_{\omega\in\Omega} X(\omega)\cdot\mathbb{P}[\{\omega\}]\geq\sum_{\substack{\omega\in\Omega\\ X(\omega)\geq a}} X(\omega)\cdot\mathbb{P}[\{\omega\}]\geq\sum_{\substack{\omega\in\Omega\\ X(\omega)\geq a}} a\cdot\mathbb{P}[\{\omega\}]=a\cdot\mathbb{P}[X\geq a].$$

Daraus folgt die Behauptung. □

Mithilfe der Markov-Ungleichung können wir beispielsweise die Wahrscheinlichkeit abschätzen, dass wir in einer Folge von n unabhängigen, fairen Münzwürfen mindestens $(3n/4)$-mal „Zahl“ werfen. Sei X die Zufallsvariable, die die Anzahl an „Zahl“ in den n Münzwürfen beschreibt. Dann ist $\mathbb{E}[X]=\frac{n}{2}$. Mit der Markov-Ungleichung folgt für $a=\frac{3}{2}$:

$$\mathbb{P}\left[X\geq\frac{3n}{4}\right]=\mathbb{P}\left[X\geq a\cdot\mathbb{E}[X]\right]\leq\frac{1}{a}=\frac{2}{3}.$$

Tatsächlich fällt die obige Wahrscheinlichkeit sogar exponentiell schnell mit n.

4.2.1 Analyse von RandMaxCut

Definition 4.16: Randomisierter Approximationsalgorithmus

Ein *randomisierter Approximationsalgorithmus* A für ein Minimierungs- oder Maximierungsproblem Π erreicht einen *Approximationsfaktor* von $r \geq 1$ bzw. $r \leq 1$, wenn

$$\mathbb{E}[w_A(I)] \leq r \cdot \mathrm{OPT}(I) \quad \text{bzw.} \quad \mathbb{E}[w_A(I)] \geq r \cdot \mathrm{OPT}(I)$$

für alle Instanzen $I \in \mathcal{I}_\Pi$ gilt. Wir sagen dann, dass A ein *randomisierter r-Approximationsalgorithmus* ist.
Oft hängt der Approximationsfaktor von der Eingabelänge ab. Dann ist $r : \mathbb{N} \to [0, \infty)$ eine Funktion und es muss

$$\mathbb{E}[w_A(I)] \leq r(|I|) \cdot \mathrm{OPT}(I) \quad \text{bzw.} \quad \mathbb{E}[w_A(I)] \geq r(|I|) \cdot \mathrm{OPT}(I)$$

für alle Instanzen $I \in \mathcal{I}_\Pi$ gelten, wobei $|I|$ die Länge der Eingabe I bezeichne.

Satz 4.17: RandMaxCut

RandMaxCut (Algorithmus 4.2) ist ein randomisierter $\frac{1}{2}$-Approximationsalgorithmus für das Maximum-Cut-Problem.

Beweis. Sei $G = (V, E)$ ein ungereichteter Graph mit Kantengewichten w. Sei $U \subseteq V$ die zufällige Teilmenge von V, die RandMaxCut auswählt. Sei X die Zufallsvariable, die das Gesamtgewicht der Kanten, die über den Schnitt laufen, angibt. Außerdem definieren wir für jede Kante $e \in E$ die Zufallsvariable X_e durch

$$X_e := \begin{cases} w(e), & \text{falls } e \text{ über den Schnitt läuft,} \\ 0, & \text{sonst.} \end{cases}$$

Dann gilt per Definition $X = \sum_{e \in E} X_e$. Mit der Linearität des Erwartungswerts (Satz 4.14 (b)) folgt

$$\mathbb{E}[X] = \mathbb{E}\left[\sum_{e \in E} X_e\right] = \sum_{e \in E} \mathbb{E}[X_e].$$

Da X_e nur die Werte 0 und $w(e)$ annehmen kann, gilt

$$\begin{aligned} \mathbb{E}[X_e] &= 0 \cdot \mathbb{P}[X_e = 0] + w(e) \cdot \mathbb{P}[X_e = w(e)] \\ &= w(e) \cdot \mathbb{P}[e \text{ läuft über den Schnitt}] \\ &= w(e) \cdot \frac{1}{2}. \end{aligned}$$

Für den letzten Schritt haben wir verwendet, dass die Wahrscheinlichkeit, dass eine Kante über den Schnitt läuft, gerade $\frac{1}{2}$ ist. Insgesamt erhalten wir somit

$$\mathbb{E}[X] = \sum_{e \in E} \mathbb{E}[X_e] = \sum_{e \in E} w(e) \cdot \frac{1}{2} = \frac{1}{2} \cdot \sum_{e \in E} w(e) = \frac{W}{2}$$

für $W = \sum_{e \in E} w(e)$. Außerdem gilt $\mathrm{OPT} \leq W$, da kein Schnitt mehr als das Gesamtgewicht aller Kanten erreichen kann. Somit folgt wie gewünscht

$$\mathbb{E}[X] = \frac{W}{2} \geq \frac{\mathrm{OPT}}{2}.$$

□

4.2.2 Binomialverteilung und geometrische Verteilung

Seien $Y_1, \dots, Y_n$ unabhängige Bernoulli-Zufallsvariablen mit demselben Parameter $p \in [0,1]$. Die Zufallsvariable $X = Y_1 + \cdots + Y_n$ heißt *binomialverteilt* mit Parametern n und p.

Satz 4.18: Binomialverteilung

Sei X eine binomialverteilte Zufallsvariable mit Parametern n und p. Dann gilt $\mathbb{E}[X] = pn$ und für alle $j \in \{0, \dots, n\}$:

$$\mathbb{P}[X = j] = \binom{n}{j} p^j (1-p)^{n-j}.$$

Beweis. Ausrechnen. □

Sei nun $Y_1, Y_2, \dots$ eine Folge unabhängiger Bernoulli-Zufallsvariablen mit demselben Parameter $p \in [0,1]$. Wir betrachten nun die Frage, wie lange wir auf den ersten Erfolg warten müssen. Diese wird durch eine *geometrisch verteilte Zufallsvariable mit Parameter p* modelliert. Eine solche Zufallsvariable X nimmt den Wert $n \in \mathbb{N}$ an, wenn $Y_1 = \cdots = Y_{n-1} = 0$ und $Y_n = 1$. Die Wahrscheinlichkeit hierfür ist $(1-p)^{n-1}p$. Wir definieren für eine geometrische Zufallsvariable X mit Parameter p dementsprechend $\mathbb{P}[X = n] = (1-p)^{n-1}p$ für alle $n \in \mathbb{N}$. Um den Erwatungswert einer geometrischen Zufallsvariablen zu bestimmen, leiten wir zunächst eine nützliche alternative Darstellung des Erwartungswerts her, die nur für Zufallsvariablen gilt, die nur Werte aus $\mathbb{N}_0$ annehmen können.

Lemma 4.19: Erwartungswert natürliche Zufallsvariable

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und sei $X : \Omega \to \mathbb{N}_0$ eine Zufallsvariable. Dann gilt

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} \mathbb{P}[X \geq i].$$

Beweis. Es gilt

$$\begin{aligned}
\sum_{i=1}^{\infty} \mathbb{P}[X \geq i] &= \sum_{i=1}^{\infty} \sum_{j=1}^{\infty} \mathbb{P}[X = j] \\
&= \sum_{j=1}^{\infty} \sum_{i=1}^{j} \mathbb{P}[X = j] \\
&= \sum_{j=1}^{\infty} j \cdot \mathbb{P}[X = j] = \mathbb{E}[X],
\end{aligned}$$

wobei wir ausgenutzt haben, dass wir bei absolut konvergenten Reihen die Reihenfolge der Summanden beliebig vertauschen können. □

Satz 4.20: Geometrische Verteilung

Sei X eine geometrisch verteilte Zufallsvariable mit Parameter p. Dann gilt $\mathbb{E}[X] = \frac{1}{p}$.

Beweis. Für die geometrische Zufallsvariable X und $j \in \mathbb{N}$ gilt

$$\mathbb{P}[X \geq j] = \sum_{i=j}^{\infty} \mathbb{P}[X = i] = \sum_{i=j}^{\infty} (1-p)^{i-1} p = (1-p)^{j-1},$$

wobei die letzte Gleichung durch Nachrechnen folgt, oder durch Überlegen, dass $X \geq j$ genau dann gilt, wenn die ersten $(j-1)$ Bernoulli-Versuche Fehlschläge sind. Mit Lemma 4.19 folgt somit

$$\mathbb{E}[X] = \sum_{i=1}^{\infty} \mathbb{P}[X \geq i] = \sum_{i=1}^{\infty} (1-p)^{i-1} = \frac{1}{1-(1-p)} = \frac{1}{p},$$

wobei wir in der letzten Gleichheit die Formel für die geometrische Reihe verwendet haben. □

Eine weitere wichtige Eigenschaft geometrisch verteilter Zufallsvariablen ist die *Gedächtnislosigkeit*. Das bedeutet, dass die Wahrscheinlichkeit eines Erfolgs unanbhängig davon ist, wie viele Misserfolge es bis jetzt gegeben hat.

Lemma 4.21: Gedächtnislosigkeit geometrischer Verteilung

Sei X eine geometrisch verteilte Zufallsvariable mit Parameter p. Dann gilt für alle $n \in \mathbb{N}$ und $k \in \mathbb{N}_0$:

$$\mathbb{P}[X = n + k \mid X > k] = \mathbb{P}[X = n].$$

Beweis. Es gilt

$$\begin{aligned}\mathbb{P}[X = n+k \mid X > k] &= \frac{\mathbb{P}[(X = n+k) \cap (X > k)]}{\mathbb{P}[X > k]} = \frac{\mathbb{P}[X = n+k]}{\mathbb{P}[X > k]} \\ &= \frac{(1-p)^{n+k-1} \cdot p}{(1-p)^k} = (1-p)^{n-1} p = \mathbb{P}[X = n].\end{aligned}$$ □

Das *Coupon Collector's Problem* tritt oft in der Analyse randomisierter Algorithmen auf und lässt sich anhant von Sammelbildern beschreiben, wie sie zum Beispiel zu jeder Fußballweltmeisterschaft verkauft werden. Wir gehen davon aus, dass es insgesamt n verschiedene solcher Bilder gibt. Diese kann man nicht gezielt erwerben, sondern man kann nur verschlossene Tüten kaufen, in denen sich jeweils unabhängig ein uniform zufällig ausgewähltes Sammelbild befindet. Wir bezeichnen mit X die Zufallsvariable, die angibt, wie viele Tüten wir kaufen müssen, bis wir jedes Bild mindestens einmal in unserer Sammlung haben.

Satz 4.22: Coupon Collector's Problem

Für die Zufallsvariable X aus dem Coupon Collector's Problem gilt $\mathbb{E}[X] = n\ln(n) + \Theta(n)$.

Beweis. Wir zerlegen den Prozess in n Phasen. Die erste Phase beginnt mit dem ersten Sammelbild, das wir kaufen. Phase i für $i \in \{2, \dots, n\}$ beginnt direkt, nachdem wir das $(i-1)$-te verschiedene Sammelbild gekauft haben, und endet, sobald wir das i-te verschiedene Sammelbild gekauft haben. Bezeichnen wir mit X_i die Anzahl an Sammelbildern, die wir in Phase i kaufen, dann gilt $X = X_1 + X_2 + \cdots + X_n$. Außerdem ist jedes X_i geometrisch verteilt mit Parameter

$$p_i = \frac{n-i+1}{n},$$

denn die Wahrscheinlichkeit, eines der $n - i + 1$ Sammelbilder zu kaufen, die nicht zu den $i-1$ bereits erworbenen Bildern gehören, beträgt in Phase i in jedem Versuch $\frac{n-i+1}{n}$. Mit der Linearität des Erwartungswertes und Satz 4.20 folgt somit

$$\mathbb{E}[X] = \sum_{i=1}^{n} \mathbb{E}[X_i] = \sum_{i=1}^{n} \frac{1}{p_i} = \sum_{i=1}^{n} \frac{n}{n-i+1} = n \cdot \sum_{i=1}^{n} \frac{1}{i} = n \cdot (\ln(n) + \Theta(1)) = n\ln(n) + \Theta(n).$$

Im letzten Schritt haben wir dabei ausgenutzt, dass für die n-te harmonische Zahl H_n gerade $\sum_{i=1}^{n} \frac{1}{i} = \ln(n) + \Theta(1)$ gilt. □

4.2.3 Vertex Cover

Wir betrachten nun einen simplen Algorithmus RandVertexCover für das Vertex-Cover-Problem. Für eine Eingabe $G = (V, E)$ startet dieser Algorithmus mit einer leeren Menge U und geht alle Kanten aus E in einer beliebigen Reihenfolge durch. Für jede Kante $e \in E$ überprüft der Algorithmus, ob bereits mindestens ein Endknoten von e in U enthalten ist. Falls ja, so macht der Algorithmus mit der nächsten Kante weiter. Ansonsten wählt er uniform zufällig einen der beiden Endknoten von e aus und fügt diesen zu U hinzu. Anschließend macht er mit der nächsten Kante weiter.

Wir betrachten sein Verhalten auf einem Sterngraphen $G = (V, E)$. Für einen solchen Graphen ist $V = \{v, u_1, \dots, u_{n-1}\}$ und die Menge E besteht nur aus den Kanten (v, u_i) für alle i. Das optimale Vertex Cover besteht nur aus dem Knoten v, der alle Kanten abdeckt. Bei jeder Kante fügt RandVertexCover mit einer Wahrscheinlichkeit von $\frac{1}{2}$ den Knoten v zur Menge U hinzu. Sobald dies einmal passiert ist, sind alle Kanten des Graphen abgedeckt und es werden keine weiteren Knoten der Menge U hinzugefügt. Sei nun X eine geometrisch verteilte Zufallsvariable mit Parameter $p = \frac{1}{2}$. Dann ist die Anzahl an Knoten, die RandVertexCover zu U hinzufügt, verteilt wie die Zufallsvariable $\min\{X, n-1\}$, da der Algorithmus maximal $|E| = n - 1$ viele Knoten zu U hinzufügen kann. Es gilt

$$\mathbb{E}[\min\{X, n-1\}] \leq \mathbb{E}[X] = \frac{1}{p} = 2 = 2 \cdot \mathrm{OPT}$$

und somit handelt es sich bei RandVertexCover bei diesem Beispiel um einen randomisierten 2-Approximationsalgorithmus.

Satz 4.23: RandVertexCover

RandVertexCover ist ein randomisierter 2-Approximationsalgorithmus für das Vertex-Cover-Problem.

Beweis. Aus der Beschreibung des Algorithmus folgt direkt, dass er stets ein Vertex Cover berechnet. Zu zeigen ist also nur, dass dieses im Erwartungswert höchstens doppelt so viele Knoten enthält wie ein minimales Vertex Cover. Sei $G = (V, E)$ eine beliebige Eingabe und sei $Z = \{z_1, \dots, z_k\} \subseteq V$ ein minimales Vertex Cover von G. Wir bezeichnen mit $V_i \subseteq V$ die Menge, die aus z_i und den zu z_i benachbarten Knoten besteht. Die Mengen V_i sind im Allgemeinen nicht paarweise disjunkt. Ferner bezeichne $E_i \subseteq E$ die Menge der zu z_i inzidentenk Kanten. Da es sich bei Z um ein Vertex Cover handelt, gilt $E = E_1 \cup \dots \cup E_k$.

Es bezeichne X_i die Anzahl an Knoten aus V_i, die RandVertexCover bei Betrachtung der Kanten aus E_i zu U hinzufügt. Es seien $e_1, \dots, e_\ell$ die Kanten aus E_i und wir nehmen an, dass RandMaxCut diese in diese Reihenfolge betrachtet. Zwischendurch betrachtet RandMaxCut im Allgemeinen auch andere Kangen aus $E \setminus E_i$. Wenn der Algorithmus den Knoten z_i bei der Betrachtung einer Kante $e_j \in E_i$ der Menge U hinzufügt, so wird bei allen weiteren Kanten aus E_i kein weiterer Knoten zu U hinzugefügt. Die Situation ist ähnlich zu obigem Sterngraphen, allerdings sind die X_i nicht identisch verteilt wie $\min\{X, |E_i|\}$ für eine geometrisch verteilte Zufallsvariable X mit Parameter $p = \frac{1}{2}$, da zwischen den Betrachtungen der Kanten aus E_i auch Kanten aus $E \setminus E_i$ betrachtet werden können, die den Zustand des Algorithmus verändern. Allerdings gilt dennoch

$$\mathbb{E}[X_i] \leq \mathbb{E}[\min\{X, |E_i|\}] \leq \mathbb{E}[X] = 2.$$

Wegen $E_1 \cup \dots \cup E_k = E$ gilt $|U| = X_1 + \dots + X_k$ und somit folgt mit der Linearität des Erwartungswerts (Satz 4.14 (*b*)):

$$\mathbb{E}[|U|] = \mathbb{E}[X_1 + \dots + X_k] = \sum_{i=1}^{k} \mathbb{E}[X_i] \leq \sum_{i=1}^{k} 2 = 2k = 2 \cdot \mathrm{OPT}\,. \qquad \square$$

4.3 Max-SAT

Die Eingabe besteht aus einer aussagenlogischen Formel in konjunktiver Normalform (KNF). Das Ziel ist es, eine Belegung der Variablen zu finden, die möglichst viele Klauseln erfüllt. In der gewichteten Variante besitzt jede Klausel ein Gewicht und das Ziel ist es, eine Belegung zu finden, die das Gesamtgewicht der erfüllten Klauseln maximiert. Aus der NP-Schwere des Entscheidungsproblems SAT folgt direkt, dass auch das Max-SAT-Problem NP-schwer ist.

Wir nennen eine Formel der Form x oder $\neg x$ für eine Variable x ein *Literal*. Ein Literal der Form x nennen wir ein *positives Literal*, ein Literal der Form $\neg x$ ein *negatives Literal*. Wir nennen die Variablen, die in der Eingabe vorkommen, im Folgenden $x_1, \dots, x_n$. Eine *Klausel* C ist eine *Disjunktion* (Oder-Verknüpfung) von Literalen über diesen Variablen und eine aussagenlogische Formel φ ist in *konjunktiver Normalform* (KNF) , wenn sie eine *Konjunktion* (Und-Verknüpfung) von Klauseln ist. Oft fassen wir Klauseln auch einfach als Menge von Literalen auf. Eine Formel in KNF ist dann von der Gestalt

$$\varphi = \bigwedge_{i=1}^{m} \left(\bigvee_{\ell \in C_l} \ell \right),$$

wobei $C_1, \dots, C_m$ die Klauseln von φ sind.

Problem 4.24: Max-SAT

Input: aussagenlogische Formel φ in KNF mit Variablen $x_1, \dots, x_n$ und Klauseln $C_1, \dots, C_m$; Gewichte $w_1, \dots, w_m \in (0, \infty)$
Output: Belegung der Variablen $x_1, \dots, x_m$
Objective: Maximiere das Gesamtgewicht der durch die Belegung erfüllten Klauseln

Genau wie schon für die Maximum-Cut- und Vertex Cover-Probleme geben wir für Max-SAT zunächst einen sehr einfachen Approximationsalgorithmus RandMaxSAT an. Dieser wählt einfach eine uniform zufällige Belegung α der Variablen $x_1, \dots, x_n$. Das bedeutet, er setzt jede Variable unabhängig von den anderen mit einer Wahrscheinlichkeit von $\frac{1}{2}$ auf 0 bzw. 1.

Satz 4.25: RandMaxSAT

RandMaxSAT ist ein randomisierter $\frac{1}{2}$-Approximationsalgorithmus für Max-SAT.

Beweis. Wir definieren zunächst für jede Klausel C_j eine Zufallsvariable

$$Y_j := \begin{cases} w_j & \text{falls } \alpha \text{ Klausel } C_j \text{ erfüllt,} \\ 0 & \text{sonst.} \end{cases}$$

Dann können wir das Gesamtgewicht Y der durch α erfüllten Klauseln als die Summe $Y_1 + \cdots + Y_m$ schreiben und es gilt

$$\mathbb{E}[Y] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j].$$

Die Klausen C_j wird nur dann nicht durch die zufällige Belegung α erfüllt, wenn alle Variablen, die als positives Literal in C_j vorkommen, auf 0 gesetzt sind und alle Variablen, die als negatives Literal in C_j vorkommen, auf 1 gesetzt sind. Es bezeichne $\ell_j = |C_j|$ die Anzahl an Literalen in C_j. Da die oben genannte Belegung der Variablen aus C_j die einzige ist, die die Klausel nicht

erfüllt, gilt

$$\mathbb{P}[\alpha \text{ erfüllt } C_j \text{ nicht}] \leq \left(\frac{1}{2}\right)^{\ell_j} \leq \frac{1}{2}.$$

Der erste Schritt ist dabei eine Ungleichung und keine Gleichung, da die Klausel C_j für eine Variable x sowohl das Literal x als auch das Literal $\neg x$ enthalten kann. Da eine solche Klausel durch jede beliebige Belegung erfüllt wird, beträgt die Wahrscheinlichkeit, dass C_j durch α nicht erfüllt wird, in diesem Fall 0. damit folgt insgesamt

$$\mathbb{E}[Y] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{j=1}^{m} w_j \cdot \frac{1}{2} = \frac{1}{2} \sum_{j=1}^{m} w_j \geq \frac{1}{2} \cdot OPT,$$

wobei die letzte Schranke daraus folgt, dass das Gesamtgewicht aller Klauseln eine obere Schranke für OPT ist. □

Damit folgt, dass der Approximationsfaktor von RandMaxSAT besser wird, je länger die Klauseln sind. Eine Klausel mit ℓ Literalen ist nur mit Wahrscheinlichkeit von höchstens $(1/2)^\ell$ nicht durch die zufällige Belegung erfüllt. Betrachten wir beispielsweise das 3-SAT-Problem, bei dem alle Klauseln genau drei Literale enthalten, so ergibt sich ein Approximationsfaktor von $\frac{7}{8}$. Es ist bekannt, dass dies sogar im Wesentlichen der beste erreichbare Approximationsfaktor für 3-SAT unter der Annahme P≠NP ist.

RandMaxCut und seine Analyse sind vollkommen analog zu dem einfachen Approximationsalgorithmus für das Maximum-Cut Problem. Wir zeigen, wie der Algorithmus *derandomisiert* werden kann. Das bedeutet, wir möchtendie eigentlich randomisierten Entscheidungnen von RandMaxSAT (d.h. die zufällige Belegung α) nun deterministisch so treffen, dass der Algorithmus den gleichen Approximationsfaktor erreicht. Danach werden wir sehen, wie wir mit einem besseren randomisierten Ansatz einen besseren Approximationsfaktor erreichen können.

4.3.1 Derandomisierung

Definition 4.26: Bedingte Erwartung

Sei $(\Omega, \mathbb{P})$ ein diskreter Wahrscheinlichkeitsraum und $A \subseteq \Omega$ ein Ereignis mit $\mathbb{P}[A] > 0$. Außerdem sei $X : \Omega \to \mathbb{R}$ eine diskrete Zufallsvariable mit Werten aus $M \subseteq \mathbb{R}$. Der *bedingte Erwartungswert von X gegeben A* existiert, falls die Reihe $\sum_{x \in M} |x| \cdot \mathbb{P}[X = x \mid A]$ konvergiert und ist dann definiert als

$$\mathbb{E}[X \mid A] = \sum_{x \in M} x \cdot \mathbb{P}[X = x \mid A].$$

Es gelten folgende Rechenregeln:

$$\mathbb{E}[X] = \mathbb{P}[A] \cdot \mathbb{E}[X \mid A] + \mathbb{P}[\overline{A}] \cdot \mathbb{E}[X \mid \overline{A}] \tag{4.1}$$

und (für B unabhängig von A)

$$\mathbb{E}[X \mid A] = \mathbb{P}[B] \cdot \mathbb{E}[X \mid A \wedge B] + \mathbb{P}[\overline{B}] \cdot \mathbb{E}[X \mid A \wedge \overline{B}]. \tag{4.2}$$

In der Derandomisierung von RandMaxSAT wählen wir die Belegung von $x_1, \ldots, x_n$ Schritt für Schritt. In Schritt i bestimmen wir die Belegung von x_i und haben davor schon die Belegung von $x_1, \ldots, x_{i-1}$ fexstgelegt. Wir betrachten zunächste den ersten Schritt, in dem wir die Belegung von x_1 festlegen. Mit Y bezeichnen wir wieder die Zufallsvariable, die das Gesamtgewicht der

erfüllten Klauseln bei einer uniform zufälligen Belegung α beschreibt. Gemäß (4.1) gilt

$$\mathbb{E}[Y] = \mathbb{P}[\alpha_1 = 0] \cdot \mathbb{P}[Y \mid \alpha_1 = 0] + \mathbb{P}[\alpha_1 = 1] \cdot \mathbb{E}[Y \mid \alpha_1 = 1] = \frac{\mathbb{E}[Y \mid \alpha_1 = 0] + \mathbb{E}[Y \mid \alpha_1 = 1]}{2}.$$

Aus dieser Gleichung folgt direkt, dass einer der beiden bedingten Erwartungswerte mindestens den Wert $\mathbb{E}[Y]$ annehmen muss. Sei $b_1 \in \{0,1\}$ mit $\mathbb{E}[Y \mid \alpha_1 = b_1] \geq \mathbb{E}[Y]$. Wir setzen x_1 nun auf b_1. Wir fahren nach dem Schema fort und betrachten Schritt i, in dem wir die Belegung von x_i festlegen und gehen davon aus, dass wir bereits Belegungen $b_1, \dots, b_{i-1}$ der Variablen $x_1, \dots, x_{i-1}$ gewählt haben, sodass gilt

$$\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}] \geq \mathbb{E}[Y].$$

Mit (4.2) gilt dann

$$\begin{aligned}
\mathbb{E}[Y] &\leq \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}] \\
&= \mathbb{P}[\alpha_i = 0] \cdot \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0] \\
&\quad + \mathbb{P}[\alpha_i = 1] \cdot \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 1] \\
&= \frac{\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0] + \mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_{i-1} = b_{i-1}, \alpha_i = 0]}{2}.
\end{aligned}$$

Mit demselben Argument muss einer der beiden bedingten Erwartungswerte mindestens den Wert $\mathbb{E}[Y]$ haben. Wir setzen x_i nun auf das b_i, für das das der Fall ist. Auf diese Weise können wir induktiv fortfahren und deterministisch eine Belegung $b_1, \dots, b_n$ der Variablen $x_1, \dots, x_n$ finden, sodass das Gesamtgewicht der erfüllten Klauseln mindestens $\mathbb{E}[Y]$ beträgt.

Wir müssen nun noch zeigen, dass dieses Verfahren effizient ausgeführt werden kann. Mit Linearität des Erwartungsertes folgt (was auch für bedingte Erwartungen gilt)

$$\mathbb{E}[Y \mid \alpha_1 = b_1, \dots, \alpha_i = b_i] = \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j \mid \alpha_1 = b_1, \dots, \alpha_i = b_i].$$

Diese bedingten Wahrscheinlichkeiten können wie folgt bestimmt werden: Erfüllt eine der Variablen $x_1, \dots, x_i$ mit der Belegung $b_1, \dots, b_i$ bereits Klausel C_j, so beträgt die bedingte Wahrscheinlichkeit 1. Ansonsten sind alle Literale in C_j, die aus $x_1, \dots, x_i$ bestehen, in der Belegung $b_1, \dots, b_i$ falsch. Diese können aus der Klausel getrichen werden. Bleiben danach noch ℓ Literale übrig, so beträgt die bedingte Wahrscheinlichkeit, dass die Klausel durch die zufällige Belegung erfüllt wird, $1 - (1/2)^\ell \geq 1/2$. (Wir nehmen an, dass nicht gleichzeitig x und $\neg x$ in der Klausel vorkommen, da diese sonst mit Wahrscheinlichkeit 1 erfüllt wäre.)

Wir erhalten insgesamt einen deterministischen $\frac{1}{2}$-Approximationsalgorithmus für Max-SAT. Die Methode der bedingten Erwartungswerte zur Derandomisierung lässt sich auch auf viele andere randomisierte Algorithmen erweitern. Insbesondere können wir mit dieser Methode auch RandMaxCut derandomisieren.

4.3.2 Verbesserter Algorithmus

Wir werden nun sehen, dass der Approximationsfaktor $\frac{1}{2}$ durch einen einfachen Trick verbessert werden kann. Wir setzen nun jede Variable mit Wahrscheinlichkeit $p > \frac{1}{2}$ unabhängig von den anderen Variablen auf 1. Für eine geeignete Wahl von p führt das zu einem besseren Approximationsfaktor. Zunächst betrachten wir nur Formeln, die keine Klauseln enthalten, die nur aus einem einzigen negativen Literal bestehen. Solche Klauseln nennen wir im Folgenden *negative Einheitsklauseln*.

Lemma 4.27: Zufällige Belegung mit Wahrscheinlichkeit p

Sei φ eine Formel in KNF über den Variablen $x_1, \dots, x_n$ mit den Klauseln $C_1, \dots, C_m$. Die Formel φ enthalte keine negativen Einheitsklauseln. Eine zufällige Belegung α der Variablen $x_1, \dots, x_n$, die jede Variable unabhängig mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ auf 1 setzt, erfüllt jede einzelne Klausel mit einer Wahrscheinlichkeit von mindestens $\min\{p, 1-p^2\}$.

Beweis. Sei C eine beliebige Klausel. Falls es sich bei C um eine Klausel der Länge 1 handelt, so besteht sie laut Voraussetzung aus einem positiven Literal. Das bedeutet, die C wird von der zufälligen Belegung mit einer Wahrscheinlichkeit von p erfüllt. Betrachten wir nun den Fall, dass es sich bei C um eine Klausel der Länge $\ell \geq 2$ handelt. Wir bezeichnen mit a und b die Anzahl an positiven bzw. negativen Literalen in C, also ist $\ell = a + b$. Enthält C für eine Variable sowohl das entsprechend positive als auch das negative Literal, ist C mit Wahrscheinlichkeit 1 erfüllt. Ansonsten gibt es genau eine Belegung der in C vorkommenden Variablen, die C nicht erfüllt: Alle positiven Literale sind auf 0 gesetzt und alle negativen Literale auf 1. Die Wahrscheinlichkeit, dass diese Belegung gewählt wird, beträgt $p^b(1-p)^a$. Folglich beträgt die Wkeit, dass C erfüllt ist,

$$1 - p^b(1-p)^a \geq 1 - p^{a+b} = 1 - p^\ell \geq 1 - p^2,$$

wobei wir $1 - p \leq p$ genutzt haben. □

Satz 4.28: Approximationsalgorithmus durch zufällige Belegung mit Wahrscheinlichkeit p

Für Max-SAT ohne negative Einheitsklauseln erhalten wir einen $\min\{p, 1-p^2\}$-Approximationsalgorithmus, indem wir jede Variable mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ unabhängig auf 1 setzen.

Beweis. Sei φ eine Formel ohne negative Einheitsklauseln und Y das Gesamtgewicht der Klauseln durch die zufällige Belegung α. Dann gilt

$$\begin{aligned}\mathbb{E}[Y] &= \sum_{j=1}^{m} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{j=1}^{m} w_j \cdot \min\{p, 1-p^2\} \\ &= \min\{p, 1-p^2\} \cdot \sum_{j=1}^{m} w_j \geq \min\{p, 1-p^2\} \cdot \mathrm{OPT}\,.\end{aligned}$$

□

$\min p, 1-p^2$ ist maximal, wenn $p = 1-p^2$. In $[0,1]$ ist das für $p = \frac{\sqrt{5}-1}{2} \approx 0.618$ der Fall.

Nun zeigen wir noch, wie wir die Annahme eliminieren können, dass es keine negativen Einheitsklauseln gibt: Sei φ eine beliebige KNF-Formel. Wir erhalten eine zu φ äquivalente Formel, indem wir für eine Variable x jedes Vorkommen von x durch $\neg x$ und jedes $\neg x$ durch x ersetzen. Das bedeutet, dass oBdA das Gewicht der Einheitsklausel x mindestens so groß ist wie das Gewicht der Einheitsklausel $\neg x$ (mit der impliziten Konvention, dass Klaoseln, die nicht in φ enthalten sind, Gewicht 0 haben). Insbesondere bedeutet das, dass nicht der Fall eintreten kann, dass φ die Einheitsklausel $\neg x$ mit positiven Gewicht, aber nicht die Einheitsklausel x enthält. Für eine Variable x_i bzeichnen wir im Folgenden mit v_i das Gewicht der negativen Einheitsklausel $\neg x$.

Lemma 4.29: OPT ist höchstens alle Gewichte minus negative Einheitsklauseln

Es gilt $\mathrm{OPT} \leq \sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i$.

Beweis. Jede Belegung erfüllt für jede Variable x_j entweder die Einheitsklausel x_j oder die Einheitsklausel $\neg x_j$. Im besten Fall erfüllt eine Belegung alle Klauseln der Länge mindestens zwei und für jede Variable x_j die Einheitsklausel mit größerem Gewicht. Aus dieser Überlegung folgt direkt das Lemma. □

Satz 4.30: randomisierter $\frac{\sqrt{5}-1}{2}$-Approximationsalgorithmus

Für MAX-SAT erhalten wir einen $\min\{p, 1-p^2\}$-Approximationsalgorithmus, indem wir jede Variable mit Wahrscheinlichkeit $p \geq \frac{1}{2}$ unabhängig auf 1 setzen. Dabei nehmen wir oBdA an, dass für jede Variable x das Gewicht der Einheitsklausel x mindestens so groß ist wie das Gewicht der Einheitsklausel $\neg x$. Für $p = \frac{\sqrt{5}-1}{2}$ ergibt sich ein Approximationsfaktor von $\frac{\sqrt{5}-1}{2} \approx 0.618$.

Beweis. Sei φ eine beliebige KNF-Formel. Sei U die Teilmenge der Klauseln von φ, die alle Klauseln aus φ bis auf die negativen Einheitsklauseln enthält. Das Gesamtgewicht der Klauseln in U beträgt $\sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i$.

Wir setzen nun jede Variable unanbhängig voneinander mit Wahrscheinlichkeit p auf 1. Sei C eine beliebige Klausel aus U. Handelt es sich bei C um eine positive Einheitsklausel, so ist sie mit Wahrscheinlichkeit p erfüllt. Enthält C für eine Variable x die Literale x und $\neg x$, so ist C mit Wahrscheinlichkeit 1 erfüllt. Ansonsten bezeichne $\ell \geq 2$ die Anzahl an Literalen in C. Ferner seien a und b die Anzahlen an positiven bzw. negativen Literalen in C. Die Wahrscheinlichkeit, dass C von der zufälligen Belegung der Variablen erfüllt wird, beträgt analog zum Argument aus Lemma 4.27

$$1 - p^b(1-p)^a \geq 1 - p^{a+b} = 1 - p^\ell \geq 1 - p^2.$$

Satz 4.30 folgt nun wieder aus der Linearität des Erwartungswertes:

$$\begin{aligned}
\mathbb{E}[Y] &\geq \sum_{C_j \in U} w_j \cdot \mathbb{P}[\alpha \text{ erfüllt } C_j] \geq \sum_{C_j \in U} w_j \cdot \min\{p, 1-p^2\} \\
&= \min\{p, 1-p^2\} \cdot \left(\sum_{j=1}^{m} w_j - \sum_{i=1}^{n} v_i \right) \\
&\geq \min\{p, 1-p^2\} \cdot \mathrm{OPT}\,.
\end{aligned}$$

□

Auch der Algorithmus aus Satz 4.30 kann mit bedingten Erwartungen derandomisiert werden.

[04.11.2025, Vorlesung 7]
[05.11.2025, Vorlesung 8]

Kapitel 5

Lineare Programmierung und Runden

[05.11.2025, Vorlesung 8]
[11.11.2025, Vorlesung 9]

Problem 5.1: Lineares Programm (LP)

Input: $c \in \mathbb{R}^d, b \in \mathbb{R}^m, A \in \mathbb{R}^{m \times d}$
Output: $x \in \mathbb{R}^d$ mit $Ax \leq b$
Objective: Minimiere/maximiere $c \cdot x$

Beispiel 5.2 (MaxFlow)**.** Sei $G = (V, E)$ ein Flussnetzwerk mit Quelle $s \in V$ und Senke $t \in V$ und Kapazitäten $c : E \to \mathbb{N}_0$. Setze x_e auf den tatsächlichen Fluss für alle $e \in E$. Dann wollen wir die Funktion

$$\sum_{e=(s,v)} x_e - \sum_{e=(v,s)} x_e$$

maximieren unter folgenden Bedingungen:

$$\begin{aligned}
&\forall e \in E : x_e \geq 0 && \text{(Fluss nicht negativ)},\\
&\forall e \in E : x_e \leq c(e) && \text{(Fluss nicht größer als Kapazität)},\\
&\forall v \in V \setminus \{s,t\} : \sum_{e=(u,v)} x_e - \sum_{e=(v,u)} x_e = 0 && \text{(Flusserhaltung)}.
\end{aligned}$$

Lineare Programme können in polynomieller Zeit gelöst werden. Im Folgenden werden *ganzzahlige lineare Programme* eine große Rolle spielen – lineare Programme, bei denen zusätzlich als Nebenbedingung gefordert ist, dass die Werte einiger oder aller Variablen ganzzahlig sein müssen. Das Finden einer solchen Variablenbelegung ist NP-schwer, was z. B. daraus folgt, dass das Rucksackproblem als ILP formuliert werden kann.

5.1 Max-SAT

Mittels LPs können wir einen besseren Approximationsfaktor erreichen.

5.1.1 Randomisiertes Runden

Sei φ eine Eingabe für Max-SAT mit Variablen $x_1, \ldots, x_n$, Klauseln $C_1, \ldots, C_n$ und Gewichten $w_1, \ldots, w_m$. Zunächst formulieren wir die Aufgabe, eine Belegung zu finden, für die das

Gesamtgewicht der erfüllten Klausen maximal wird, als ILP:

$$\begin{aligned} \max & \sum_{j=1}^{m} w_j z_j \\ s.t. & \sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j, && \forall j \in \{1, \dots, m\}, \\ & y_i \in \{0, 1\}, && \forall i \in \{1, \dots, n\}, \\ & 0 \leq z_j \leq 1, && \forall j \in \{1, \dots, m\}, \end{aligned}$$

wobei $z_j = 1$ genau dann gelten soll, wenn die durch $y_1, \dots, y_n$ codierte Belegung die Klausel C_j erfüllt. Mit P_j und N_j sind die Indizes der Variablen gemeint, die positiv bzw. negativ in C_j vorkommen.

Wir haben nun das Problem, eine optimale Variablenbelegung zu finden, auf das Lösen eines ILP reduziert. Dies ist auch NP-schwer. Wir werden dieses ILP aber *relaxieren*, d.h. die Bedingung, dass nur bestimmte Werte angenommen werden dürfen, aufgeben:

$$\begin{aligned} \max & \sum_{j=1}^{m} w_j z_j \\ s.t. & \sum_{i \in P_j} y_i + \sum_{i \in N_j} (1 - y_i) \geq z_j, && \forall j \in \{1, \dots, m\}, \\ & 0 \leq y_i \leq 1, && \forall i \in \{1, \dots, n\}, \\ & 0 \leq z_j \leq 1, && \forall j \in \{1, \dots, m\}, \end{aligned}$$

Im Folgenden seien Z^*_{LP} und Z^*_{ILP} die optimalen Zielfunktionswerte des LP bzw. des ILP. Es gilt dann $Z^*_{ILP} = \text{OPT}$, wobei OPT das Gesamtgewicht der Klauseln angibt, die in einer optimalen Belegung der Aussagenvariablen $x_1, \dots, x_n$ erfüllt sind. Es gilt außerdem offensichtlich $Z^*_{LP} \geq Z^*_{ILP}$.

Es bezeichne (y^*, z^*) mit $y^* = (y^*_1, \dots, y^*_n)$ und $z^* = (z^*_1, \dots, z^*_n)$ eine optimale Lösung des linearen Programms. Der Algorithmus RandomizedRounding sieht wie folgt aus:

Algorithmus 5.3 RandomizedRounding

Input: Wie in Problem 4.24
Output: Wie in Problem 4.24

1: Löse das relaxierte LP
2: Setze jede Aussagenvariable x_i mit Wahrscheinlichkeit y^*_i auf 1 und sonst auf 0

Satz 5.4: RandomizedRounding

RandomizedRounding ist ein randomisierter $\left(1 - \frac{1}{e}\right)$-Approximationsalgorithmus für Max-SAT.

Beweis. Wir betrachten wieder für jede Klausel separat die Wahrscheinlichkeit, dass sie durch die zufällige Belegung der Variablen erfüllt wird. Mit obiger Notation erhalten wir für jede Klausel C_j

$$\mathbb{P}[C_j \text{ nicht erfüllt}] = \prod_{i \in P_j} (1 - y^*_i) \cdot \prod_{i \in N_j} y^*_i. \tag{5.1}$$

Dies folgt daraus, dass es nur eine Belegung der in C_j vorkommenden Variablen gibt, die C_j nicht erfüllt: Alle positiv auftretenden Variablen auf 0, alle negativ auftretenden auf 1. Die rechte Seite beschreibt genau die Wahrscheinlichkeit, dass diese Belegung gewählt wird. Wir verwenden die

AM-GM-Ungleichung $\prod_{i=1}^{n} a_i \leq \left(\frac{1}{n}\sum_{i=1}^{n} a_i\right)^n$ für alle $a_1, \ldots, a_n \in \mathbb{R}_{\geq 0}$. Angewendet auf (5.1) ergibt sich mit $|C_j| = |P_j| + |N_j|$

$$\begin{aligned}\prod_{i\in P_j}(1-y_i^*)\cdot\prod_{i\in N_j} y_i^* &\leq \left(\frac{1}{|C_j|}\left(\sum_{i\in P_j}(1-y_i^*)+\sum_{i\in y_i^*}\right)\right)^{|C_j|}\\ &= \left(\frac{1}{|C_j|}\left(|C_j|+\sum_{i\in P_j}(-y_i^*)+\sum_{i\in N_j}(y_i^*-1)\right)\right)^{|C_j|}\\ &= \left(1-\frac{1}{|C_j|}\left(\sum_{i\in P_j} y_i^*+\sum_{i\in N_j}(1-y_i^*)\right)\right)^{|C_j|}\end{aligned}$$

und mit der Nebenbedingung

$$\sum_{i\in P_j} y_i^* + \sum_{i\in N_j}(1-y_i^*) \geq z_j,$$

die das LP garantiert, ergibt sich insgesamt

$$\mathbb{P}[C_j \text{ nicht erfüllt}] \leq \left(1-\frac{z_j^*}{|C_j|}\right)^{|C_j|}$$

und dementsprechend

$$\mathbb{P}[C_j \text{ erfüllt}] \geq 1-\left(1-\frac{z_j^*}{|C_j|}\right)^{|C_j|}.$$

Betrachte die Funktion $f(x) = 1-\left(1-\frac{x}{|C_j|}\right)^{|C_j|}$ genauer. Die zweite Ableitung auf $[0,1]$ ist für jedes $|C_j| \geq 1$ nicht positiv, folglich ist die Funktion konkav und die Gerade g, die durch die Punkte $(0, f(0))$ und $(1, f(1))$ verläuft, liegt an keiner Stelle über der Funktion f. Wegen $f(0) = 0$ entspricht g der Funktion

$$g(x) = f(1)\cdot x = \left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot x$$

und es gilt insgesamt

$$\mathbb{P}[C_j \text{ erfüllt}] \geq f(z_j^*) \geq g(z_j^*) = \left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot z_j^*.$$

Bezeichnen wir wieder mit Y das Gesamtgewicht der erfüllten Klauseln, so ergibt sich mit der Linearität des Erwartungswertes

$$\begin{aligned}\mathbb{E}[Y] &= \sum_{j=1}^{m}\mathbb{P}[C_j \text{ erfüllt}] \geq \sum_{j=1}^{m} w_d\left(1-\left(1-\frac{1}{|C_j|}\right)^{|C_j|}\right)\cdot z_j^*\\ &\geq \min_{k\geq 1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot\sum_{j=1}^{m} w_j z_j^* = \min_{k\geq 1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot Z_{LP}^*\\ &\geq \min_{k\geq 1}\left(1-\left(1-\frac{1}{k}\right)^k\right)\cdot \mathrm{OPT}\,.\end{aligned}$$

Die Funktion $1-\left(1-\frac{1}{k}\right)^k$ ist streng monoton fallend und geht von oben gegen $1-\frac{1}{e}$. Damit

erhalten wir

$$\mathbb{E}[Y] \geq \left(1 - \frac{1}{e}\right) \cdot \mathrm{OPT}\,. \qquad \square$$

Es gilt $\left(1 - \frac{1}{e}\right) \approx 0.632$ und somit erreicht RANDOMIZEDROUNDING einen etwas besseren Approximationsfaktor als die Algorithmen aus Kapitel 4.3. Kombinieren macht es aber noch besser:

5.1.2 Kombination beider Algorithmen

Betrachten wir RANDMAXSAT und RANDOMIZEDROUNDING genauer, so stellen wir fest, dass der Approximationsfaktor von RANDMAXSAT besser wird, je länger die Klauseln wird, und der Approximationsfaktor von RANDOMIZEDROUNDING besser wird, je kürzer die Klauseln sind. Konkret gilt für jede Klausel, dass sie durch RANDOMIZEDROUNDING mit einer Wahrscheinlichkeit von mindestens $\left(1 - \left(1 - \frac{1}{|C_j|}\right)^{|C_j|}\right) \cdot z_j^*$ erfüllt wird und durch RANDMAXSAT von mindestens $1 - 2^{-|C_j|}$.

Algorithmus 5.5 BETTEROFTWO

Input: Wie in Problem 4.24
Output: Wie in Problem 4.24

1: Führe RANDOMIZEDROUNDING aus
2: Führe RANDMAXSAT aus
3: Gib die bessere Belegung zurück

Satz 5.6: BetterOfTwo

BETTEROFTWO ist ein randomisierter $\frac{3}{4}$-Approximationsalgorithmus für MAX-SAT.

Beweis. Wir bezeichnen mit Y_1 und Y_2 das Gesamtgewicht der durch RANDOMIZEDROUNDING und RANDMAXSAT erfüllten Klauseln. Da BETTEROFTWO beide Algorithmen ausführt und anschließend das bessere Ergebnis wählt, interessieren wir uns für den Erwartungswert von $\max\{Y_1, Y_2\}$. Es gilt

$$\mathbb{E}[\max\{Y_1, Y_2\}] \geq \mathbb{E}\left[\frac{Y_1 + Y_2}{2}\right] = \frac{\mathbb{E}[Y_1] + \mathbb{E}[Y_2]}{2}.$$

Einsetzen der Infos aus den Beweisen von Sätzen 5.4 und 4.25 liefert

$$\begin{aligned}\frac{\mathbb{E}[Y_1] + \mathbb{E}[Y_2]}{2} &\geq \frac{1}{2}\sum_{j=1}^{m} w_j \left(1 - \left(1 - \frac{1}{|C_j|}\right)^{|C_j|}\right) \cdot z_j^* + \frac{1}{2}\sum_{j=1}^{m} w_j \left(1 - 2^{-|C_j|}\right) \\ &\geq \sum_{j=1}^{m} w_j z_j^* \left(\frac{1}{2}\left(1 - \left(1 - \frac{1}{|C_j|}\right)^{|C_j|}\right) + \frac{1}{2}\left(1 - 2^{-|C_j|}\right)\right).\end{aligned}$$

Wir betrachten nun die Funktion

$$f(x) = \frac{1}{2}\left(1 - \left(1 - \frac{1}{x}\right)^{x}\right) + \frac{1}{2}\left(1 - 2^{-x}\right).$$

Diese Funktion nimmt für alle ganzzahligen Werte mindestens den Wert $\frac{3}{4}$ an:

$$f(1) = \frac{1}{2} + \frac{1}{2} \cdot \frac{1}{2} = \frac{3}{4} \quad \text{und} \quad f(2) = \frac{1}{2}\left(1 - \frac{1}{4}\right) + \frac{1}{2}\left(1 - \frac{1}{4}\right) = \frac{3}{4}.$$

Für $x \geq 3$ gilt

$$f(x) \geq \frac{1}{2}\left(1 - \frac{1}{e}\right) + \frac{1}{2}\left(1 - 2^{-3}\right) \approx 0.753 \geq \frac{3}{4}.$$

Damit gilt insgesamt

$$\mathbb{E}[\max\{Y_1, Y_2\}] \geq \frac{3}{4}\sum_{j=1}^{m} w_j z_j^* = \frac{3}{4} \cdot Z_{LP}^* \geq \frac{3}{4} \cdot \mathrm{OPT}\,. \qquad \square$$

5.1.3 Nichtlineares randomisiertes Runden

Algorithmus 5.7 RandRounding$_f$

Input: Wie in Problem 4.24, Funktion $f : [0,1] \to [0,1]$
Output: Wie in Problem 4.24

1: Löse das relaxierte LP
2: Setze jede Aussagenvariable x_i mit Wahrscheinlichkeit $f(y_i^*)$ auf 1 und sonst auf 0
3: ▷ *Für $f(x) = x$ ist dies einfach* RandomizedRounding. ◁

Wir analysieren den Approximationsfaktor von RandRounding$_f$ für eine beliebige Funktion $f : [0,1] \to [0,1]$ mit

$$1 - 4^{-x} \leq f(x) \leq 4^{x-1} \tag{5.2}$$

für alle $x \in [0,1]$. Wir werden sehen, dass damit jede Klausel C_j mit Wahrscheinlichkeit von mindestens $\frac{3}{4}z_j^*$ erfüllt ist.

Satz 5.8: RandRounding$_f$

Für jede Funktion f mit Eigenschaft (5.2) ist RandRounding$_f$ *ein randomisierter $\frac{3}{4}$-Approximationsalgorithmus für* Max-SAT.

Beweis. Wir zeigen für jede Klausel, dass sie mit Wahrscheinlichkeit von mindestens $\frac{3}{4}z_j^*$ erfüllt ist. Der Satz folgt dann aus dieser Aussage analog zum Beweis von Satz 5.4.

Sei C_j eine beliebige Klausel. Wieder bezeichnen P_j und N_j die Mengen der Variablen, die positiv bzw. negativ in C_j vorkommen. Die Klausel C_j ist nur dann nicht erfüllt, wenn alle positiv auftretenden Variablen auf 0 und alle neagtiv auftretenden Variablen auf 1 gesetzt sind. Daraus folgt

$$\mathbb{P}[C_j \text{ nicht erfüllt}] = \prod_{i \in P_j}(1 - f(y_i^*)) \cdot \prod_{i \in N_j} f(y_i^*) \leq \prod_{i \in P_j} 4^{-y_i^*} \cdot \prod_{i \in N_j} 4^{y_i^* - 1}$$

durch (5.2). Es gilt mit der Ungleichung

$$\sum_{i \in P_j} y_i^* + \sum_{i \in N_j}(1 - y_i^*) \geq z_j^*$$

aus dem LP gerade

$$\prod_{i \in P_j} 4^{-y_i^*} \cdot \prod_{i \in N_j} 4^{y_i^* - 1} = 4^{-\left(\sum_{i \in P_j} y_i^* + \sum_{i \in N_j}(1 - y_i^*)\right)} \leq 4^{-z_j^*}.$$

Es gilt somit

$$\mathbb{P}[C_j \text{ erfüllt}] \geq 1 - 4^{-z_j^*}.$$

Da die Funktion $g(x) = 1 - 4^{-x}$ in $[0,1]$ konkav ist, ist die Gerade h, die durch die Punkte $(0, g(0))$ und $(1, g(1))$ geht, eine untere Schranke (analog zum Beweis von Satz 5.4). Wegen

$g(0) = 0$ gilt $h(x) = g(1) \cdot x$ und somit insgesamt

$$\mathbb{P}[C_j \text{ erfüllt}] \geq 1 - 4^{-z_j^*} = g(z_j^*) \geq h(z_j^*) = g(1) \cdot z_j^* = \left(1 - \frac{1}{4}\right) z_j^* = \frac{3}{4} z_j^*.$$

Wieder bezeichnen wir mit Y das Gesamtgewicht der erfüllten Klauseln. Insgesamt ergibt sich

$$\begin{aligned}
\mathbb{E}[Y] &= \sum_{j=1}^{m} w_j \cdot \mathbb{P}[C_j \text{ erfüllt}] \\
&\geq \sum_{j=1}^{m} w_j \cdot \frac{3}{4} z_j^* = \frac{3}{4} \sum_{j=1}^{m} w_j z_j^* \\
&= \frac{3}{4} \cdot Z_{LP}^* \geq \frac{3}{4} \cdot \mathrm{OPT}\,.
\end{aligned}$$

□

Wir fragen uns nun, ob ein besserer Approximationsfaktor als $\frac{3}{4}$ erreicht werden kann.

Beispiel 5.9 (Instanz für Max-SAT)**.** Wir betrachten die Formel

$$\varphi = (x_1 \vee x_2) \wedge (x_1 \vee \neg x_2) \wedge (\neg x_1 \vee x_2) \wedge (\neg x_1 \vee \neg x_2),$$

wobei alle Klauseln das Gewicht 1 haben. Man sieht leicht, dass jede Belegung der Aussagenvariablen genau drei der Klauseln erfüllt. Somit gilt $\mathrm{OPT} = 3$. In der optimalen Lösung des LP gilt $y_1^* = y_2^* = \frac{1}{2}$ und $z_j = 1$ für alle Klauseln. Folglich ist $Z_{LP}^* = 4$.

Es gibt also Eingaben φ für Max-SAT für die $\mathrm{OPT} \leq \frac{3}{4} Z_{LP}^*$ gilt, also können wir keinen besseren Approximationsfaktor als $\frac{3}{4}$ zeigen, wenn wir als obere Schranke nur Z_{LP}^* verwenden. Dafür haben wir aber folgende Verallgemeinerung:

Definition 5.10: Integrality Gap

Das *Integrality Gap* ist der Quotient aus dem optimalen Wert des ILPs und dem optimalen Wert des LPs, das durch Relaxierung des ILPs entsteht.
Haben wir für ein Optimierungsproblem eine Formulierung der Instanzen als ILP gefunden, so bezeichnen wir mit dem *Integrality Gap dieser Formulierung* den das schlechteste Integrality Gap einer Instanz des Optimierungsproblems.

Bei Maximierungsproblemen betrachten wir also das kleinste Integrality Gap über alle Instanzen und bei Minimierungsproblemen das größte.

Allgemein gilt, dass wir für einen Approximationsalgorithmus keinen besseren Approximationsfaktor als das Integrality Gap zeigen können, wenn wir als einzige Schranke für den Wert einer optimalen Lösung die optimale Lösung des relaxierten LPs benutzen.

[11.11.2025, Vorlesung 9]
[12.11.2025, Vorlesung 10]

[12.11.2025, Vorlesung 10]
[18.11.2025, Vorlesung 11]

[18.11.2025, Vorlesung 11]
[19.11.2025, Vorlesung 12]

Anhang

Liste der Definitionen

1.1 Optimierungsproblem, Approximationsalgorithmus . . . 3
2.1 Vertex Cover . . . 4
2.2 Matching . . . 4
2.5 Set Cover . . . 5
2.9 Scheduling auf identischen Maschinen . . . 6
2.16 Rucksackproblem (Knapsack Problem, KP) . . . 8
3.1 Approximationsschema, PTAS, FPATS . . . 10
3.10 Bin-Packing . . . 13
3.15 Stark NP-schwer . . . 15
4.1 Maximum-Cut Problem . . . 17
4.3 Wahrscheinlichkeitsraum . . . 17
4.5 Unabhängige Ereignisse . . . 18
4.7 Bedingte Wahrscheinlichkeit . . . 18
4.9 Zufallsvariable . . . 19
4.11 Unabhängigkeit (Zufallsvariablen) . . . 19
4.12 Erwartungswert . . . 19
4.16 Randomisierter Approximationsalgorithmus . . . 22
4.24 Max-SAT . . . 26
4.26 Bedingte Erwartung . . . 27
5.1 Lineares Programm (LP) . . . 31
5.10 Integrality Gap . . . 36

Liste der Aussagen

2.4 Matching-VC 4
2.7 Greedy-SC 5
2.8 Preis in Greedy-SC 6
2.11 LeastLoaded 7
2.14 LPT 7
2.18 Greedy-KP 8
3.4 DynKP 11
3.6 FPTAS-KP 11
3.9 GroßKleinScheduling 12
3.11 Bin-Packing 14
3.13 $B_\varepsilon(T)$ 15
3.14 PTAS Scheduling 15
3.16 Scheduling hat keinen FPTAS 15
4.14 Linearität des Erwartungswerts 20
4.15 Markov 21
4.17 RandMaxCut 22
4.18 Binomialverteilung 23
4.19 Erwartungswert natürliche Zufallsvariable 23
4.20 Geometrische Verteilung 23
4.21 Gedächtnislosigkeit geometrischer Verteilung 24
4.22 Coupon Collector's Problem 24
4.23 RandVertexCover 25
4.25 RandMaxSAT 26
4.27 Zufällige Belegung mit Wahrscheinlichkeit p 29
4.28 Approximationsalgorithmus durch zufällige Belegung mit Wahrscheinlichkeit p . 29
4.29 OPT ist höchstens alle Gewichte minus negative Einheitsklauseln 29
4.30 randomisierter $\frac{\sqrt{5}-1}{2}$-Approximationsalgorithmus 30
5.4 RandomizedRounding 32
5.6 BetterOfTwo 34
5.8 RandRounding$_f$ 35

Liste der Algorithmen

2.3 Matching-VC 4
2.6 Greedy-SC 5
2.10 LeastLoaded 6
2.13 LongestProcessingTime (LPT) 7
2.17 Greedy-KP 8
3.3 DynKP 11
3.5 FPTAS-KP 11
3.8 GrossKleinScheduling 12
3.12 $B_\varepsilon(T)$ 14
4.2 RandMaxCut 17
5.3 RandomizedRounding 32
5.5 BetterOfTwo 34
5.7 RandRounding$_f$ 35

Index

Approximationsalgorithmus, 3
    derandomisierter -, 27
    randomisierter r-, 22
Approximationsfaktor, 22
Approximationsschema, 10
    polynomielles -, 10
    voll-polynomielles -, 10

bedingte Wahrscheinlichkeit, 18
bedingter Erwartungswert, 27
BetterOfTwo, 34
Bin-Packing, 13
Binomialverteilung, 23

Coupon Collector's Problem, 24

derandomisiert, 27
Disjunktion, 26
DynKP, 11

Elementarereignis, 17
Ereignis, 17
    Gegen-, 17
    unabhängige -se, 18
Ergebnismenge Ω, 17
Erwartungswert, 19
    bedinger -, 27

FPATS, 10
FPTAS-KP, 11

Gedächtnislosigkeit, 24
geometrische Verteilung, 23
    Gedächtnislosigkeit, 24
Greedy-KP, 8
Greedy-SC, 5

Integrality Gap, 36

Klausel, 26
    negative Einheits-, 28
Knapsack-Problem, 8, 10
Konjunktion, 26
konjunktive Normalform, 26
KP, 8, 10

LeastLoaded, 6
Lineares Programm (LP), 31
    ganzzahliges - (ILP)
        Integrality Gap eines -es, 36
    ganzzaliges - (ILP), 31
    relaxiertes -, 32
Literal, 26
    negatives -, 26
    positives -, 26
LongestProcessingTime, 7
LPT, 7

Matching, 4
    inklusionsmaximales -, 4
Matching-VC, 4
Max-SAT, 26
MaxCut, 17

negative Einheitsklausel, 28

PTAS, 10

RandRounding_f, 35
RandMaxCut, 17
RandMaxSAT, 26
RandomizedRounding, 32
RandVertexCover, 25
relaxieren, 32
Rucksackproblem, 8, 10

Scheduling, 6, 12
    auf identischen Maschinen, 6, 12
Set Cover, 5
σ-additiv, 17
stark NP-schwer, 15

Unabhängigkeit, 19
    von Zufallsvariablen, 19

Vertex Cover, 4

Wahrscheinlichkeit, 17
    bedingte -, 18
Wahrscheinlichkeitsmaß, 17
Wahrscheinlichkeitsraum, 17
    Elementarereignis, 17
    Ereignis, 17
        unabhängige -se, 18
    Ergebnismenge Ω, 17
    Produktraum, 18
    σ-additiv, 17
    Wahrscheinlichkeit, 17
    Wahrscheinlichkeitsmaß, 17

Zufallsvariable, 19
binomialverteilte -n, 23
Erwartungswert, 19
bedingter -, 27
geometrisch verteilte -n, 23
Gedächtnislosigkeit, 24
unabhängige -n, 19